INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

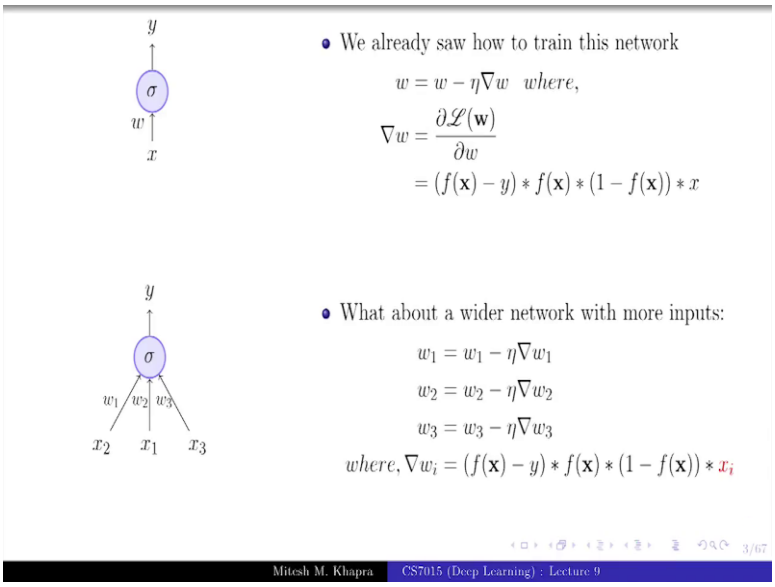
A quick recap of training deep neural networks
Lecture – 09
Greedy Layerwise Pre-training, Better activation functions, Better weight initialization methods, Batch normalization

Welcome to lecture 9 of CS7015. Today we will talk about Greedy layer wise pre-training, better activation functions, better weight initialization methods, and batch normalization. So, today's lecture is more like tips and tricks to make deep learning work.

So, when you are actually experimenting with deep learning in practice, what are some of the things that you need to take keep in mind. And it is also my way of connecting the history that we saw to where we are today right. So, there were certain things which we saw in the history, and now I will try to bring those back and connect to where we are headed from here right; where we have reached today and where we are headed from here.

So, that is with that in module 1, I will do a very quick recap of training neural networks and not take more than 5 minutes and I need it for a specific purpose.

(Refer Slide Time: 01:06)



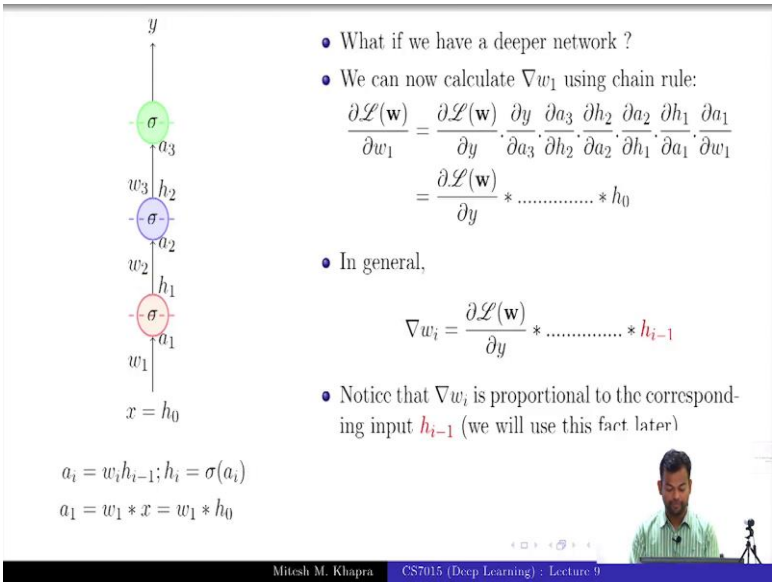
So, we already saw how to train such a very shallow neural network, what was the learning algorithm gradient descent and this was the update rule right. In particular, I wanted you to notice that the gradient actually depends on the input.

So, when you compute the gradient formula you have this multiplication by x. So, it is proportional to the input and this is one fact that we will use it; at least a couple of cases in the lecture today. So, this was a very shallow single neuron network, what if we have a wider network; Still which algorithm?

Student: Gradient descent (Refer Time: 01:40).

Gradient descent ok and we just have these 3 different formulae. And for each of these formulae note that the gradient or rather this gradient depends on the input that you are feeding in ok, I did not keep this in mind.

(Refer Slide Time: 01:52)



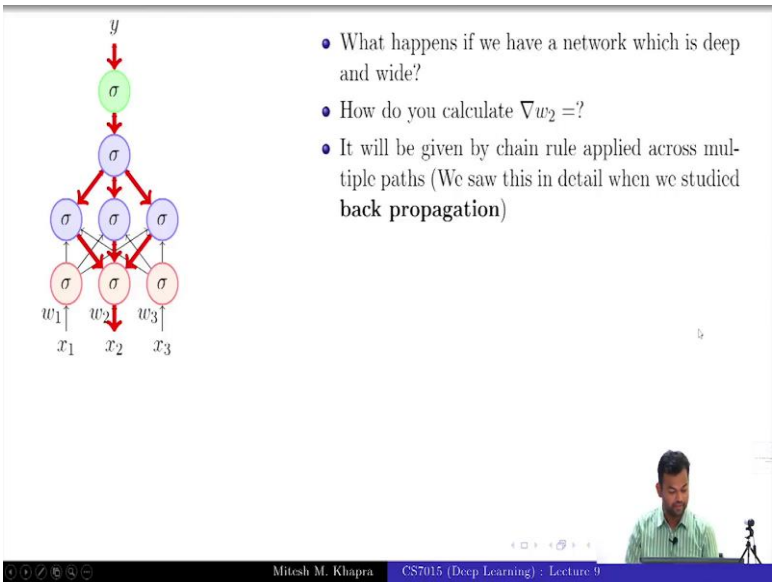
And what if you have a deeper network? So, we saw a very shallow network, we saw a wide network and I am showing you a deep network what will you do? Again gradient descent.

But you will apply the chain rule for computing the gradients. And again here in general you will notice that for any of these weights w1, w2, w3 the gradient formula will have this h i minus 1, what is h i minus 1?

Student: (Refer Time: 02:17).

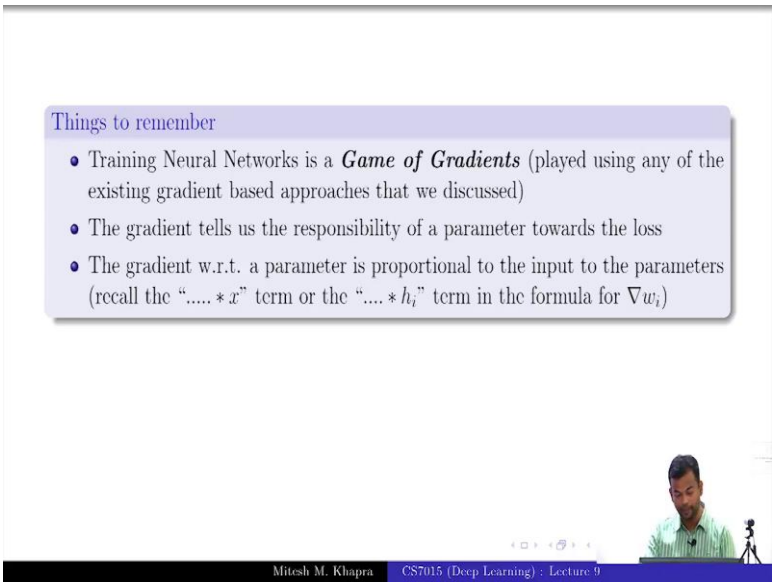
Input from the previous layer right, and h0 is the actual input. So, the gradient at any layer is actually proportional to the input from the previous layer and this could either be the input from the hidden layer or the actual input.

(Refer Slide Time: 02:34)



And finally, we saw this thin. So, we saw a wide network, we saw a thin network. Now we will see a wide network and a deep network right, sorry we saw earlier we saw a wide network and a deep network; now we see a wide and deep network. And here again you have compute the gradient by applying this chain rule across multiple paths and that is what we use and we call it back propagation. And remember again they are the same thing holds that the gradients at some point are proportional to the input at that layer everyone remembers that ok?

(Refer Slide Time: 03:07)

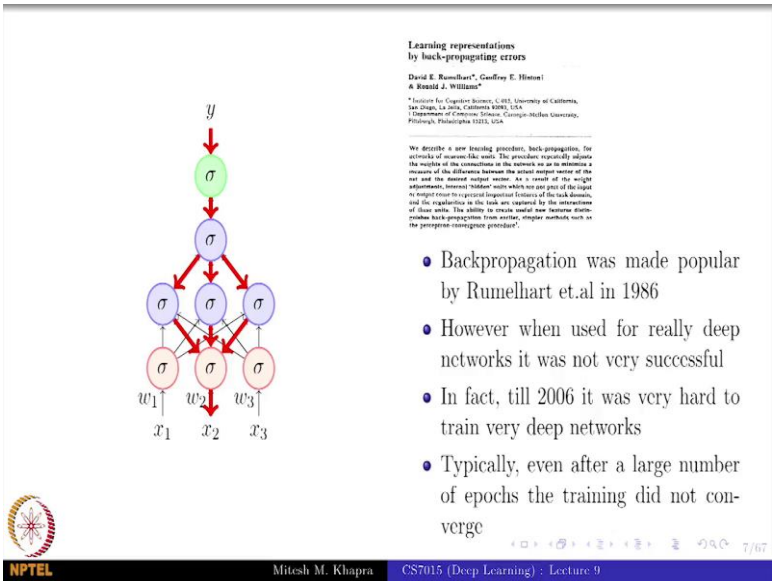


So, this is important. So, what we have is things to remember from, what we have seen so far is that; so, training neural networks is basically a game of gradients right. So, you compute the gradients and everything depends on those how will you update the weights and everything from there on is about the gradients.

And these gradients actually tell you the responsibility of the parameters towards the loss, and you appropriately update them. And we saw a variant way different sorry various variants of how to use the gradient. So, we saw the gradient descent, we saw nag momentum and all.

But in all of these the underlying core thing was to compute the gradient and then do some manipulations based on that. And the other key thing is that, the gradient at a particular layer depends on the input to that layer ok?

(Refer Slide Time: 03:48)



So now let us go back and just retrospect a better and see what is it that we have learned so far. So, so far what I have taught you gradient descent oh sorry, backpropagation, is something which was proposed way back in 1986 right?

So in fact, it was existing before that, but it was popularized by this work of Rumelhart and others in 1986 right. So, but then in the 1990s or early 2000; if back propagation already existed and we could train deep neural networks, then why did not we here so much about deep learning at that time? Of course, you guys were busy with school and all at that time, but why did the others or older people like me not hear about it?

Student: Computational power.

Computational power is that the only thing?

Student: (Refer Time: 04:35).

Computation and memory is are the only thing?

Student: Convergence

Who said convergence? Ok good. So, actually what happened right in the late 80s and early 90s and even early 2000, when you used back propagation to train really deep networks, it was not very successful. And what do I mean by not successful? Actually, what are the 2 things that could happen someone gave the answer already.

Student: (Refer Time: 05:00).

It does not converge right; that means, you do not reach the optimum solution right. In fact, till 2006 it was very hard to train very deep networks.

And typically even a after a large number of epochs these networks did not converge; that means, they were still at a very high loss and although in principle everything is fine, you have a deep neural network, you have an algorithm that can train it, but you are still not being able to train it properly and you are not being able to make any practical use of that.

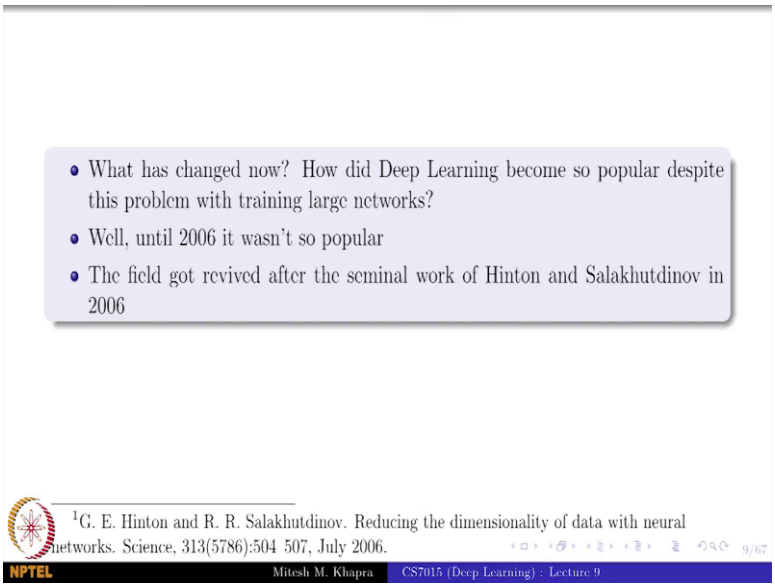
So, that was the story till 2006. So, today is about what happened in 2006? What it led to in the next few years and then where we are currently, right? So, that is the journey that we need to make ok. And that is why, we started off with this quick recap of back propagation because that is what I want to tell you that why did it not work earlier and where are we today.

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 9.2
Lecture – 09
Unsupervised pre-training

So, with that we go on to the next module, in which we will talk about Unsupervised pre training.

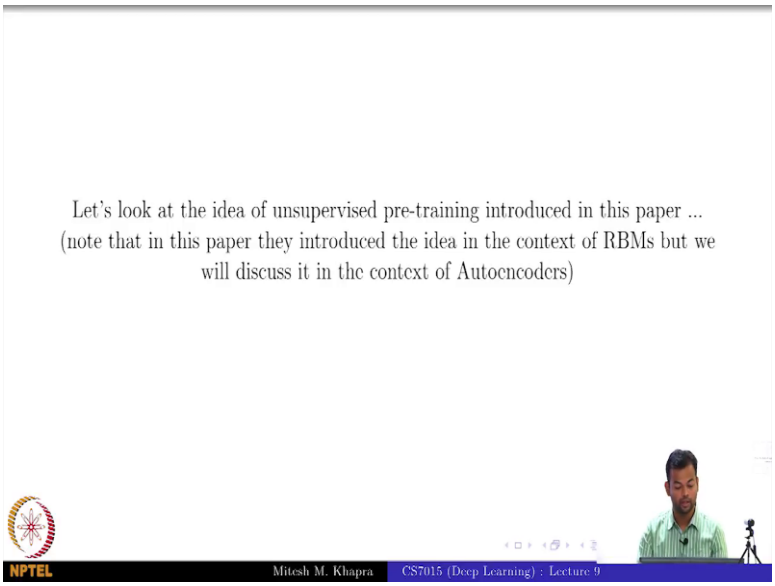
(Refer Slide Time: 00:20)



So, this work which I am going to talk about they trying to understand what has changed since the late 90s or the early 2000. How did deep learning become so popular despite this problem with training them right this problem was there?

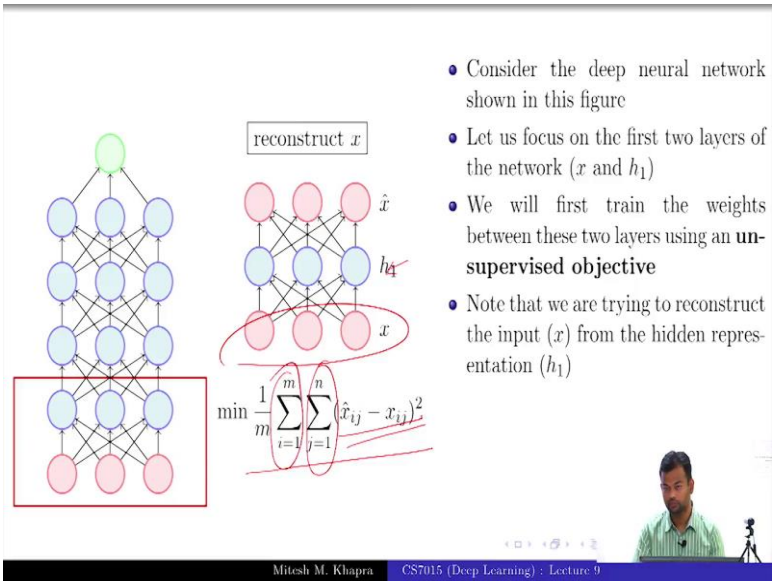
So, what happened to them solve it right. And this field actually got revived by this seminal work by Hinton and others in 2006.

(Refer Slide Time: 00:46)



So, let us see what that idea was? So, this is the idea of unsupervised pre training. In the original paper they introduce idea in the context of something known as R B M's, which we will do in the last 33 percent of the course. But we could do the same with auto encoders which we have already done. So, in this lecture I am going to talk about this idea in the context of auto encoders.

(Refer Slide Time: 01:08)



So consider the deep neural network shown in this figure. So, the a module name and the idea was unsupervised pre training. So, that itself is a giveaway of what is going to

happen, ok? So, suppose this is the deep neural network that I have designed for a particular classification task. So, what it is doing is this taking an input which is the red colored neurons that you see at the input it has 4 hidden layers; that means, it is 4 layer deep. And then you have the output layer, which tells you whether positive or negative right. That is the network that I have and I know that this is hard to train such a network the loss will not converge and I will not get anything meaningful.

So, what these guys suggested is that forget about the supervised criteria that you have; that means, you are trying to minimize a classification loss just forget about that just take the first 2 layers of this network ok which is x and h 1 right. So, you take the original input x. You feed it to some transformations and you get the hidden representation h 1 and now try to reconstruct x from h 1, what is this?

Student: Auto encoder.

Auto encoder ok, what is the objective of the auto encoder?

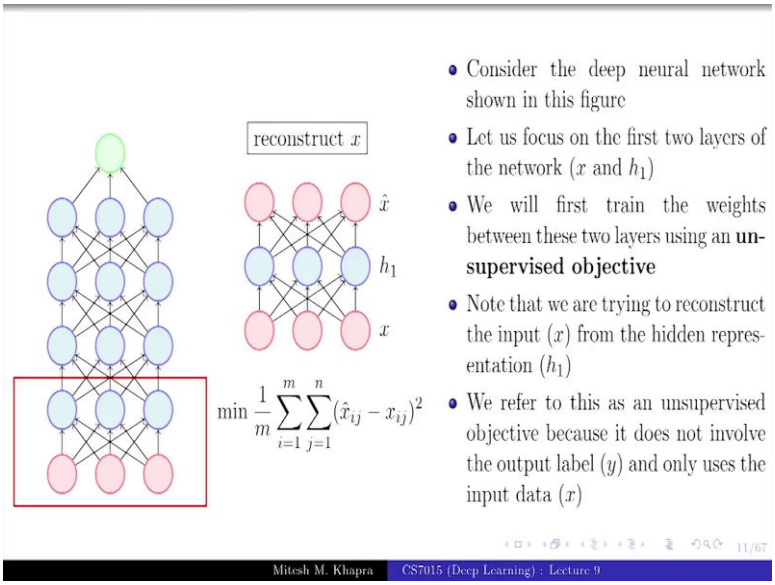
Student: (Refer Time: 02:19).

It is exactly this, for each of the m training examples look at each of the dimensions of your input and minimize the square difference between the actual input and the predicted input right, is that fine. That is what an auto encoder does. So, this is what they suggested ok. So, right now I am not telling you why this makes sense and all that that is what we will do later. Right now I am just telling you the trick then we will analyze by that trick works. And why is this objective unsupervised?

Student: (Refer Time: 02:52).

Because we are not using any labels we just giving an input and we just reconstructing the input we only have x's we do not have y's of course, eventually we will use the y, but at this stage when I am calling it unsupervised pre training I am not using the y.

(Refer Slide Time: 03:10)



Now, at the end of this, what would happen, yeah what would h 1 learn?

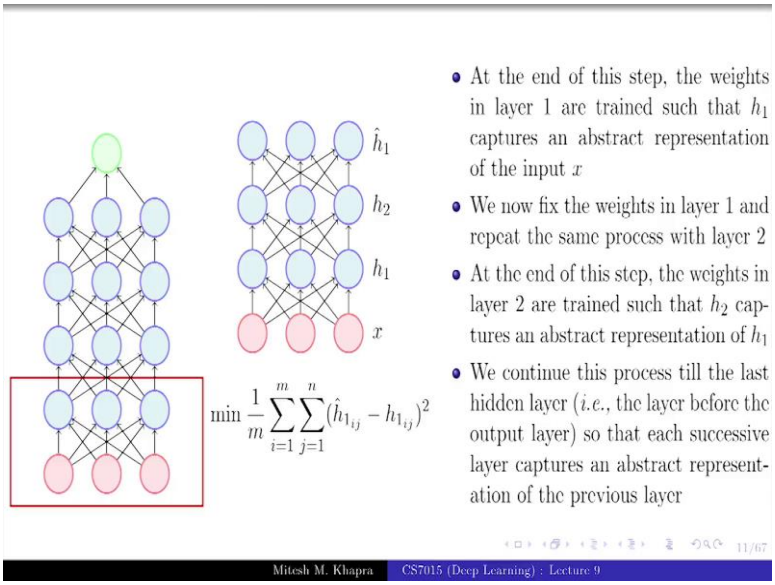
Student: (Refer Time: 03:19).

It will learn an abstract representation of x, was that our original task? What were we interested in.

Student: (Refer Time: 03:29).

In the classification task, but we are doing something very different, why we will see ok. Now guess what would the next step be? Does this make sense?

(Refer Slide Time: 03:36)



Now, at the end of the first unsupervised pre training, I have ensured that h 1 which is this layer has learned some abstract representation of the input right, and that I know from the auto encoder I mean the auto encoder which we have learned earlier right that at learns an abstract representation of the input.

Now, I have this so; that means, given an input, I know how to compute an extract representation and I am also sure that it captures the important characteristics of the data. I will just repeat this process I know that I have 4 layers in my original network. So, I will now take h 1 try to compute h 2 and then reconstruct h 1 from it. So, the in effect what am I doing in plain English, learning and even more.

Student: (Refer Time: 04:19).

Abstract representation of the input, h 1 was already one abstract representation now from this I am learning an even more abstract representation and does the objective function makes sense right. All I have done is replaced x by h 1 right. In both these places the rest of it is the same for all the training examples for all the dimensions. And throughout I am assuming that we are n layers, I mean sorry, n neurons and every layer including the input layer.

Now, what would the next step be? fix the weights in h 1 layer fix the weights in it is 2 layer and now try to reconstruct h 2 from this h 2 right. And in this way we will continue

and learn all the hidden representations. Does that look ok right? So, at least this much we believe it because we know that auto encoder works and you are just using an auto encoder and we are using it incrementally, from every abstract representation learn an even more abstract representation.

Now, at the end of this what will I do, what was my original task?

Student: classification.

Classification. So, what will I do?

Student: (Refer Time: 05:23).

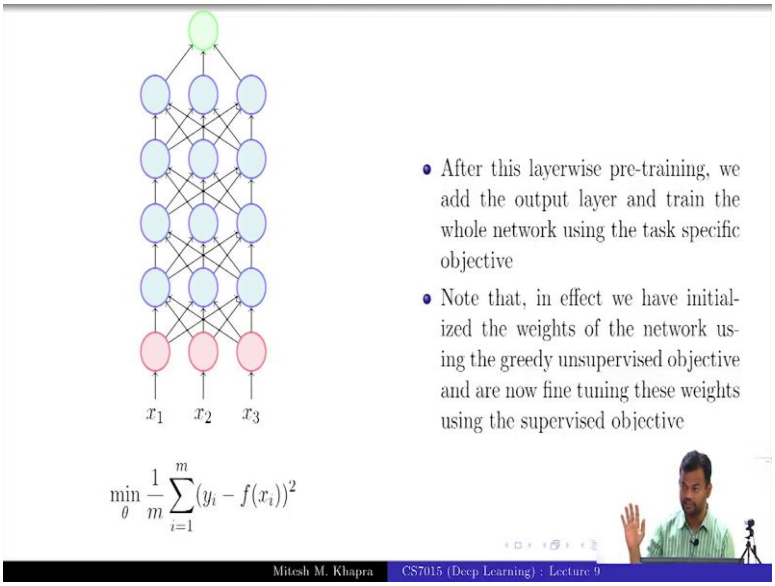
What is a network that I have, when I finish this unsupervised pre training?

Student: (Refer Time: 05:29).

No, tell me of the diagrams that you see on the slide, how much of the network would I have right? Everything except the green output layer right because the last step would be take h 4 or sorry, take h 3 and reconstruct h 3 from it and in the process learn h 4 right is that clear.

So, I would have learnt till that point. And now what I am going to do is something very simple.

(Refer Slide Time: 05:59)



I will after this layerwise pre training is done, I will add my output layer now all the weights in my network for every layer have been initialized.

And they have been initialized in a way that, that layer learns a good abstract representation of the input right. That is the thing that we have achieved at the end of unsupervised pre training that every layer has learned and more and more abstract representation of the input right. Now I will keep all these weights initialized to whatever I learned in the pre training setup does that make sense.

So; that means, instead of taking this big network with the output layer and initializing the way it is randomly, I am just going to use whatever weights I learned using the unsupervised pre training ok. So, can you tell me what has happened in terms of the error surface and so on or my movement in the w b plane or in this case, this very high dimensional w plane.

I have reached some configuration for the w's, where I know that each of these layers is a good meaningful representation of my original input right. Is that a fair statement in English, how many of you agree with this ah? Anyone has any questions at this point, one layer weights that is what you do in answer because if you train all the four then you are again entering the same problem which you had earlier right?

You cannot back you cannot back propagate through all the 4 layers because now it is a deep network and we know that does not work. So, at every layer you fix whatever you have learned so far. And at a time you are training only one layer. So, that is one interesting way of looking at it right you know that the deep neural network with 4 layers was not trainable.

So now we have reduced it to one layer at a time, I knew that one layer at a time works right is that fine. Now I will add the output layer and what will I do train the weights of the

Student: Output layer (Refer Time: 07:53).

I will not just do that I will fine tune the entire network; that means, I will train the weights of the output layer. And I will also fine tune the entire network, but now I am

contradicting myself I just gave an answer to him, that again I am doing this deep training and I know that deep training does not work.

But this actually works do you get the difference right. One is that when I start from I take this big network I start from random weight initialization and try to train it that is the story from 1986 to 2006 that in most cases these networks did not converge.

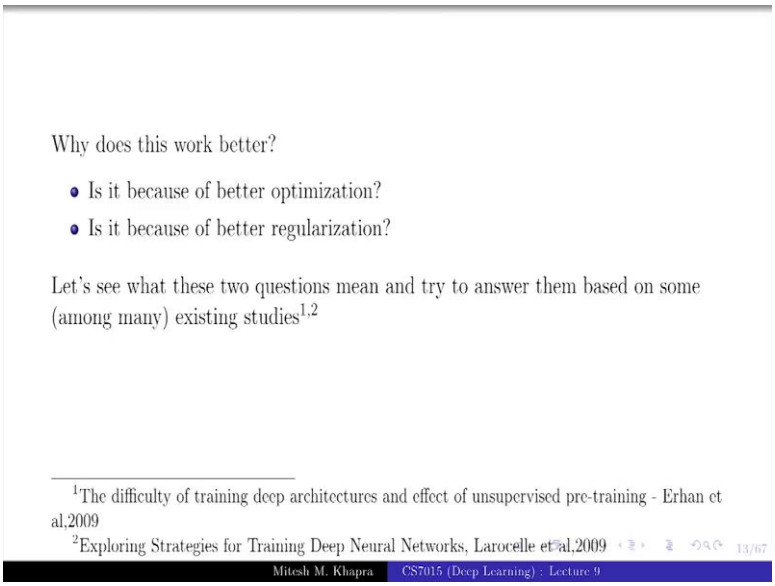
So now in 2006, we came up or someone came up with this idea of unsupervised pre training where you train the layer network one layer at a time, you do up till the last layer now you add the output layer and then fine tune the entire network; that means, back propagate over the entire network is a set up clear to everyone how many you understand the setup.

Now, again when I am doing the last step, which is known as fine tuning I have to back propagate over the entire network because I am saying I will adjust all the weights, but suddenly this works, as compared to starting from scratch. You see the problem and you see why this is important then because this has now given you a way of training deep neural network I still not told you, why it works.

We will delve into it, but not really give any concrete answers because concrete answers do not exist, but we will at least try to get some intuitions behind why it works. So, you get the setup now that this is what was happening till 1986 to 2006. And now with this idea suddenly deep neural networks were being able to train well.

So, in effect what we have done is, we have initialized the weights of the network using the unsupervised objective right. So now, initial starting with random weights, we have some weights which cater to the unsupervised objective that we had and the unsupervised objective was us layer wise reconstruction. So, that is what has happened in plain English is that fine everyone gets that?

(Refer Slide Time: 09:55)



Now, the question is, why does this work better. And I give you 2 options and I want to think about both these options ok. Is it because of better optimization or is it because of better generalization no that is not an option, but I of course, we will relate it to that, but given these 2 I want you to think whether there is any difference between these 2 statements or not, that is the first thing I want you to see. How many if you get the difference between these 2 statements, not many why is it so? What is optimization deal with dash data or dash data.

Student: (Refer Time: 10:35).

The answer you can give dash right, dash 1 data or dash 2 data what is optimization deal with?

Student: Training data.

Training data optimization remains on training data what does generalization depend on?

Student: It as 0.

It as 0 so, you get the difference between these 2 questions fine. So, let us try to answer this again here right this is 2006 to 2009 period that I am going to talk about. There are some answers and just bear with me I will give you those answers some of them will not look very convincing, but what happened after that or as a result of these investigations,

that is more important right whether these answers make sense or not, they will make sense to an extent I am not saying that we will just be bluffing.

But it will not be very convincing because there is no theory behind it right. So, what is convincing if I give you a proof that this less this is equal to that right then if we give you a proof and everything you do not have any other questions, that is not what I am going to give you. I am going to give you some intuitions, because that is all these existing works from 2006 to 2009, had and then I will make a commentary on that which will lead us to some other things. So, just bear with me for a few minutes right.

Student: (Refer Time: 11:46).

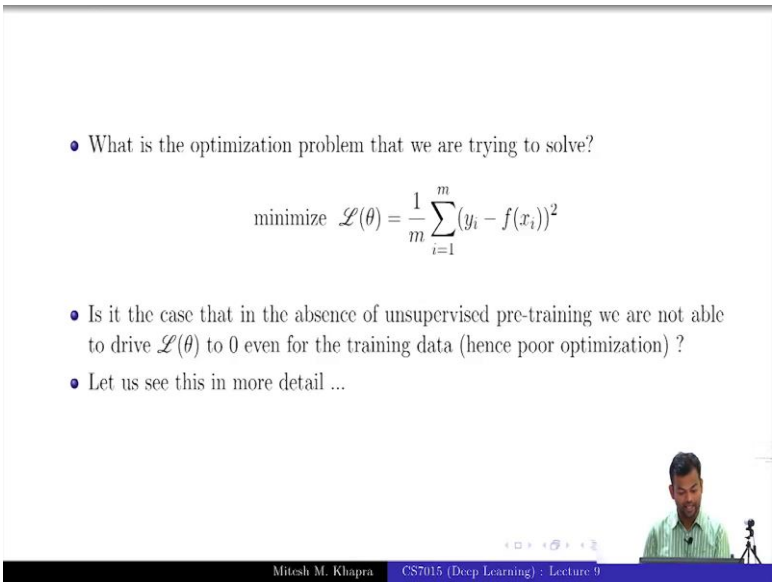
That is the optimization problem if that was the case the, I will just come to that that is what I want to talk about ok. So, it is. So, these are the 2 questions that we are dealing with right, and the answer is depends. So, we will see what it is.

(Refer Slide Time: 11:58)



So, let us first examine the case when it is because of better optimization.

(Refer Slide Time: 12:03)



So, let us first understand what is the meaning of this question, when I ask is it because of better optimization then the question that I am asking you is that, the first set up where I was trying to train everything from scratch, compared to the second set up where I had this unsupervised pre training, is it that the optimization problem becomes easier in the second set up. Now if the optimization problem becomes easier what do I actually mean by that that I was able to drive the dash to dash.

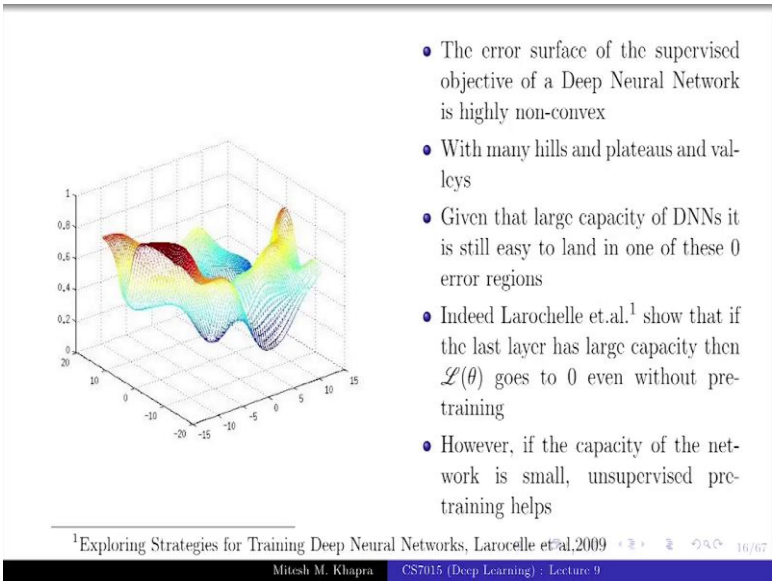
Student: Loss to 0.

Loss to 0 right. So, is it that this is the optimization problem that we were interested in.

So, is it the case that in the absence of unsupervised pre training, we are not able to drive the loss to 0 for the training data and hence poor optimization right that if; you do not do this unsupervised pre training even for the training data we cannot drive at loss to 0; that means, our optimization problem itself is not working properly right I mean the problem is fine.

But the solution is not good you get that, do you understand what is the subtle meaning of this? How many if you get this? So, let us see this in more detail right?

(Refer Slide Time: 13:03)



So, the error surface of the supervised objective of a deep neural network is highly non convex it looks something like this or even nastier than this. And in particular it has many hills and plateaus and valleys we saw this even in the toy examples that, we were dealing with right. And given the large capacity of deep neural networks it is still easy to land in one of these 0 error regions, on what basis am I making the statement which theorem?

Student: (Refer Time: 13:32).

Universal approximation theorem that is what the universal approximation theorem told us. In fact, there is a study the paper which has been cited. It showed that if the last year has a very large capacity then you can drive the loss to 0 even without pre training. Do you get the meaning of this? What does is mean? So, I have the input I have a series of hidden layers what do I mean by the last layer has a lot of capacity, what do I mean by that? It has a lot of dash.

Student: Parameters.

Parameters now how do I create these parameters I will just grow the size of the last hidden layer right. And using that then I will predict this one y.

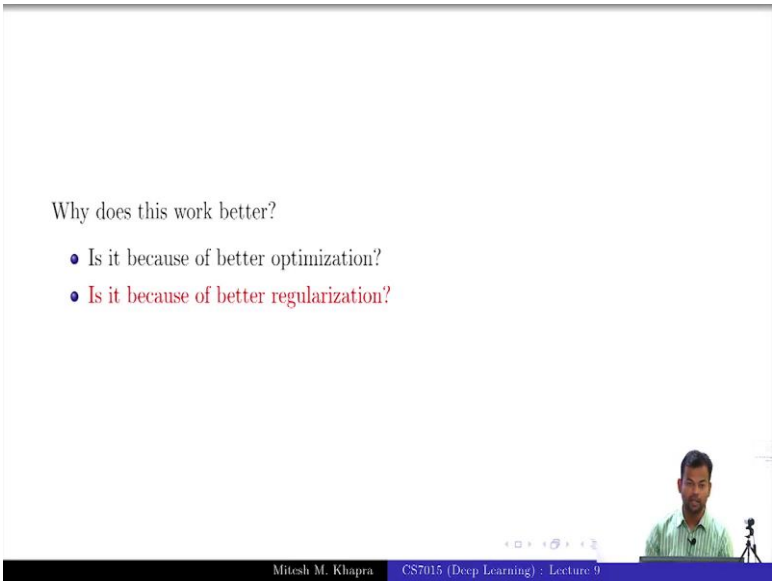
So, so that is how I could increase so that is exactly what they did they took a very deep neural network and made sure that the last layer was given a very high capacity, and then

they showered that even if you do not do an unsupervised pre training, you can still drive the training loss to 0, right?

So, this was hinting that maybe this is not an optimization problem this is something it is still not very conclusion, but we will just go with these studies we will just all I am saying is that do not shoot the messenger this is what the study says I am just relaying it back to you right. And they will have questions on these which will try to address, but if the capacity of the network is small then the unsupervised pre training helps.

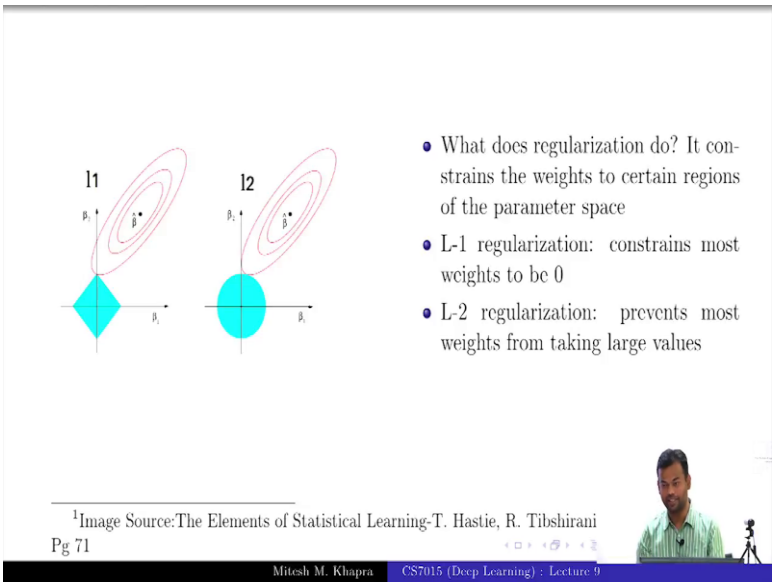
So, if you do not have these large capacity networks, but you have very deep networks, in that case unsupervised pre training helps and this is all empirical observation right there is no proof which says that given a capacity k with so much error bound I can guarantee that the loss would be epsilon within the 0 loss and so on. It nothing like that that is what it should have been ideally the case in which case life is much easier for me, but that is not the case this is just an empirical study as are most of the studies done in the period of 2006 to 2009.

(Refer Slide Time: 15:30)



So, that tells us something about what optimization means and whether this was an optimization problem or not.

(Refer Slide Time: 15:38)

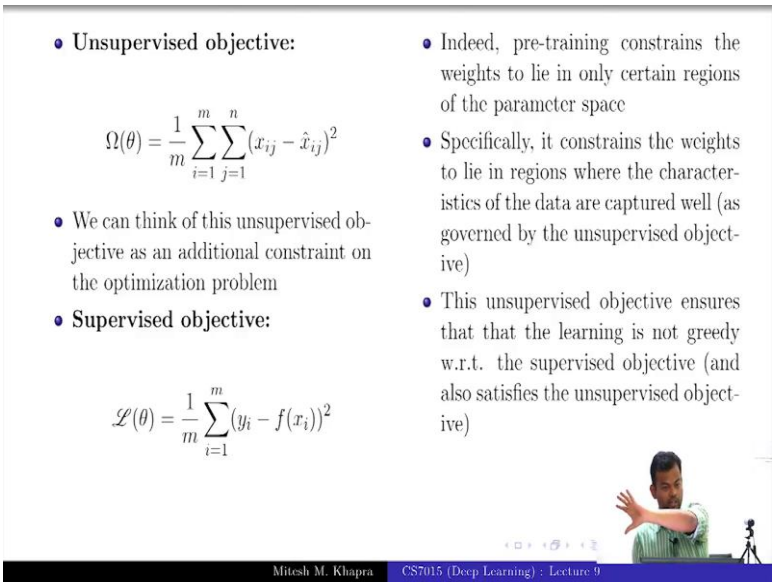


So, let us look at the other question is it because of better regularization. So, what does regularization do or you gave the exact answer it constrains the weights to lie between lie in some regions. So, it does not allow the weights a lot of freedom right? And so, you know what l one regulation does it constrains the weights to this box and l 2 regularization constrains us to this circle why no why this I know this, but why?

Student: (Refer Time: 16:01).

In why the circle I am pretty sure most of you do not know what you are saying, but you are saying the right answers, but anyways I will test this in the quiz. So, I have given you another quiz question on camera. So, yeah so a prevents a loss from taking large values.

(Refer Slide Time: 16:18)



So, indeed pre training also constrains to the way to lie in certain regions of the parameter space, why am I making this statement? What is the meaning of the statement?

So, I told you that what regularization does and from there I am making this jump and saying that even with pre training the same thing happens that your weights are actually constrained to certain regions of the parameter space, why am I making this statement ? And what are these regions that the weight is constrained to? Think l theta think omega theta. Any regulation is of the form l theta plus omega theta.

Let us see so it constrains the way to lie in regions where the characteristic of the data are captured well, that is what unsupervised pre training does it is trying to train the network in a way that each layer actually captures the important characteristics of the data, and this is based on our understanding and belief in auto encoders. So, you could actually think of this that the unsupervised objective that you had for all these layers that was actually omega theta, you are first trying to optimize omega theta.

So, in a normal regulation problem you put l theta and omega theta together and then you try to balance them, but here you have done it slightly differently you first gave it omega theta, which is the lost of reconstruction and you asked it to minimize this loss across for every layer.

Student: (Refer Time: 17:48).

No is this fine tuning. So now, what; that means, is that see remember that this is a very high dimensional region. Where you initialize makes a lot of difference. So, with this unsupervised pre training you are at least ending up in reason. So, you could think of it as a constraint that ok, move wherever you want to, but start from here which automatically means that I have I mean I have how to it is some other regions in that parameter space you get that.

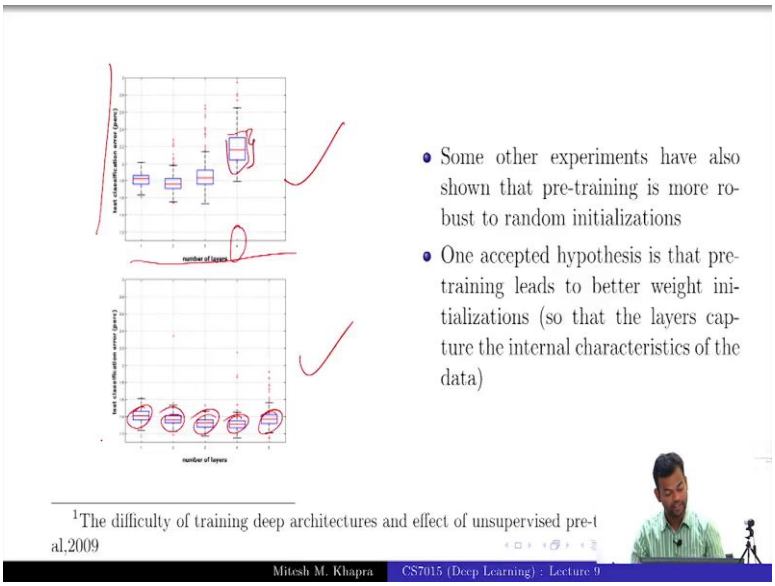
Student: (Refer time: 18:16).

As you typically that would be one thing. And it would also mean that you are starting from there. So, with this early stopping and other criteria you will not be able to grow much out from here right. So, just if that makes sense geometrically from here, you would not be able to move all the way there you get that everyone gets this question and the answer.

So, you see what the answer per is object was and you also see the difference between a normal regularization and this regularization, in the normal case you had l theta plus omega theta put together and then you are trying to minimize the sum of these 2. It was a joint optimization here you have first done omega theta ensured that the weights that you learn minimize this objective. And now you add in the supervise objective which is L theta, right?

So now this makes sure that your network cannot be too greedy with respect to L theta because it has been constrained, that has to first honor the omega theta because that is where you started and now from there on it has to decide how to do L theta, does that make sense? You see how this is acting as a regularizer is that ok? And that links back to your weight initialization thing right fine.

(Refer Slide Time: 19:20)



So, some other experiments have also shown that pre training is more robust to random initializations.

Now, what do I mean that mean by that. So, in these 2 graphs that you see here. So, this on the x axis you have the number of layers that you add to your deep neural network. And on the y axis you have the error that your network gives, when you try different initializations right. So, this box actually tells you the variance in the error.

So; that means, I tried training a network with 4 layers and I tried different initializations and the error varied in this range ok, is that good or bad? What would we want typically something which looks like the plot below right, where all these variances are little; that means, even once you do unsupervised pre training, right? It is more robust to random initializations random initializations of what?

Student: (Refer Time: 20:15).

The original random initializations from which point you started the unsupervised pre training ok, because once you have done the unsupervised pre training that is your initialization, everyone gets that?

(Refer Slide Time: 20:29)



So, these are some let us see ok. So, these are some empirical studies and let me just make a comment on these.

So, what happened from 2006 to 2009 is people showed that see this is possible you can actually train a deep neural network, using some of these tricks. We do not have a very clear answer for why this works and you could argue different way. So, this is optimization this is regularization and so on, but I do not have any theory supporting it there is no proof for why unsupervised pre training works all of these are empirical observations

But what it at least established was that it is possible to do this. So now, if it is possible to do this let me see if there are better ways of doing this, do we actually need to do unsupervised pre training, oh I think it is better regularization then why not I try better regularization techniques and see whether that helps. So, that led to the evolution of which thing that you have already seen yeah which regularization technique that you saw in the last class.

Student: Drop out.

Drop out right. So, drop out was something specific to neural networks which was introduced in the context of neural networks. So, this is because people started believing it is possible. So, let us try even better ways of doing that. So, that is how dropout came

out right? Then people said maybe optimization is the problem maybe these earlier algorithms, which up till that point was which algorithm.

Student: (Refer Time: 21:48).

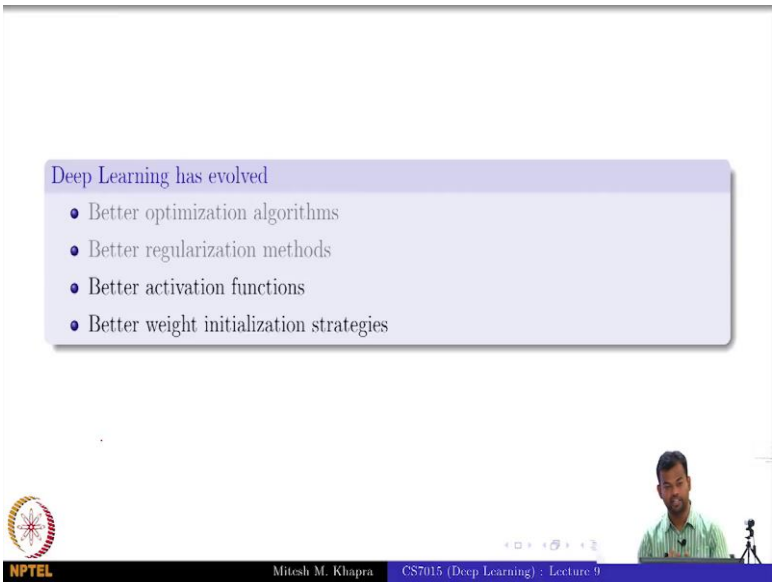
Gradient (Refer Time: 21:49) maybe that was not good. So, let us try to decide and design better optimization methods and that led to the evolution of Adam, AdaGard, RMSprop so on right so although these studies were not. So, theoretical in what they were trying to prove, they created this hope which then led to a lot of prolific work in that field right? So, at least you get the context now right the some of these might look oh this is one data set people did experiments on m l s, but I could have taken a different data set and showed that these results do not hold and so on. You could always ask those questions.

But at least what happened is people started believing these and people started questioning that ok, unsupervised pre training is one thing what else can I do. And now what has eventually happened is today no one uses unsupervised pre training right that method which led to the revival of this field, and you would have hoped that that would actually survive for many years that is out.

Now, hardly anyone uses unsupervised pre training it is only used in the context of transfer learning. So, what I mean by that is that if you have a model trained for one classification say classification of images on one data set right?

Now, you have a very small amount of data in some other domain. So, instead of training a network from scratch for this domain you will just initialize it with the weights for whatever you have trained on data set one. So, that is more of transfer learning rather than unsupervised pre training. So, that is still very prevalent, but this reliance on unsupervised training to make sure that the network actually trains that is largely phased out.

(Refer Slide Time: 23:13)



Because what has happened since 2006 and 2009, is that we have better optimization algorithms which are RMSprop, Ada grad, Adam even. So, on right many various and even now that research area is active as I was saying just in December there was a paper which pointed out some flaws in Adam and how to improve it and so on. We are better regularization methods the most prominent among those being.

Student: Dropout.

Dropout. So, these 2 are things which you have already seen today we are going to talk about better activation functions this is again something which evolved that maybe sigmoid tanh are not good. So, maybe something else is needed and then better weight initialization strategies. So, then people took this inference oh one way of looking at unsupervised pre training is that it actually initializes the weights in a better way from where on it becomes easier for me to reach convergence.

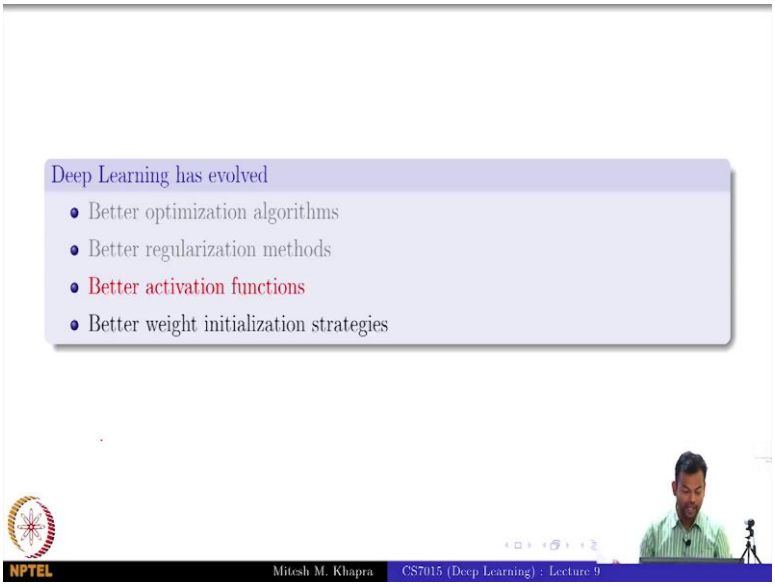
So, why do not i come up with better weight initialization methods itself, instead of relying on this indirect way of initializing the weights. So, you get this. So, get the whole picture, now what we have been doing in the past few lectures and how it connects to the history and these studies which were done from the period 2000 to 2009. How many if you get the whole picture ok? So, that is where we are now. So, today we are going to talk about better activation functions and better weight initialization methods.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

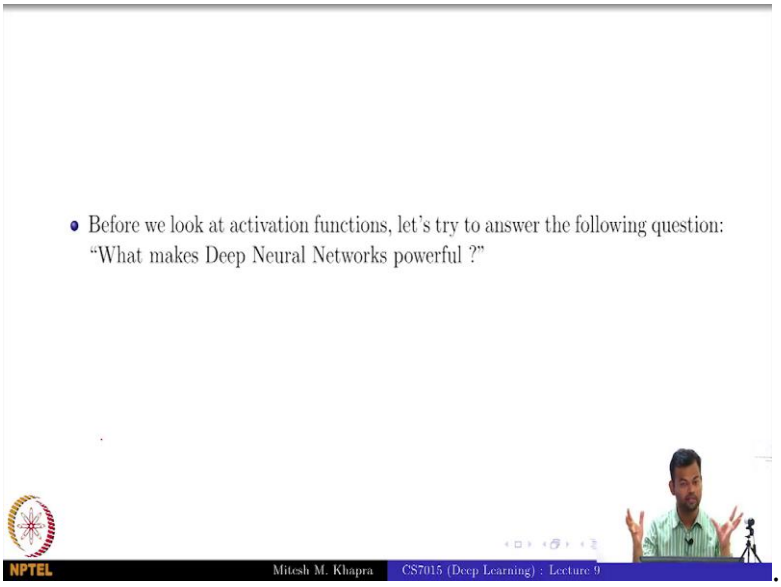
Module – 9.3
Lecture – 09
Better activation functions

Let us start with Better activation functions.

(Refer Slide Time: 00:16)

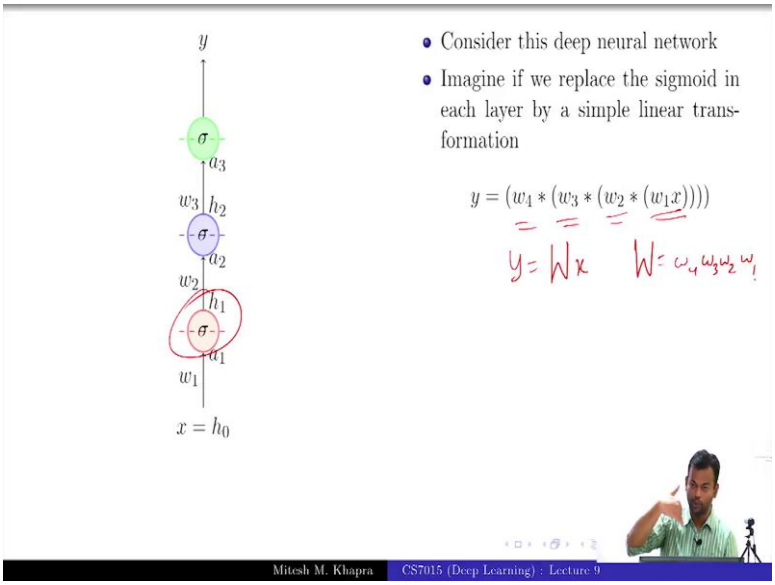


(Refer Slide Time: 00:17)



So, before I get into activation functions right, let me first tell you why I care about activation functions. Why do I actually want to come up with better activation functions so, will start with the following question. What makes deep neural networks powerful among other things? What is this one thing which makes it powerful? So, let me give you this intuition.

(Refer Slide Time: 00:34)



This I have a deep network ok, and do not worry it is a thin network, but I could have had a wide network also, but just for illustration I have taking a deep network thin network

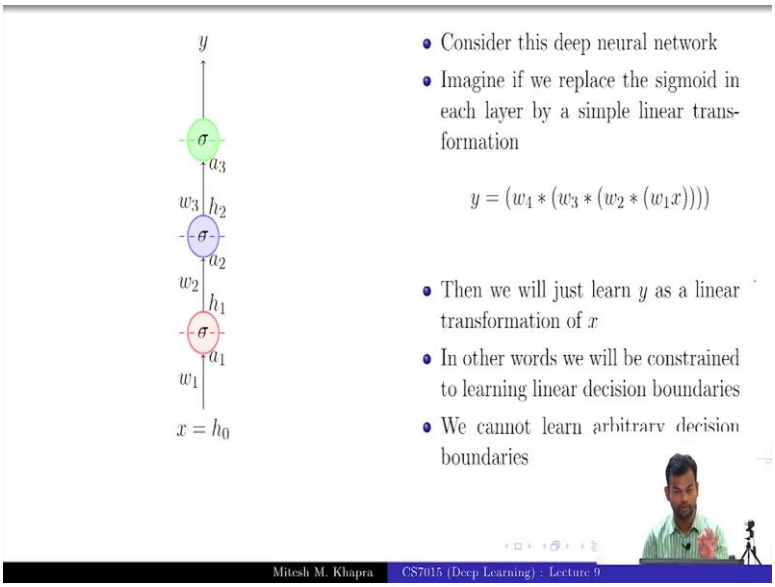
Now imagine that, each of these neurons that you have. If I replace the sigmoid in each layer by a simple linear transformation a by the way this is technically incorrect. So, orange is always input. So, this should not be a sigmoid there right, either add one more layer there or let us change the figure.

So, suppose I replace all these Sigmoids by linear transformations, what would y be can you write? Y as a function of x. What would it be? Give me the function? Will we just be this right? So, first we will do w 1 of x which is this right, then will take w 2 of that then w 3 of that and w 4 of that.

So, I could actually have written this just as y equal to W x where W is equal to w 4 w 3 w 2 w 1. So, there is no depth here, there is actually only one weight which I could have

learned you get that right? If you just have all linear transformations, then essentially you do not have so many weights, you just have one weight throughout you get that make sense.

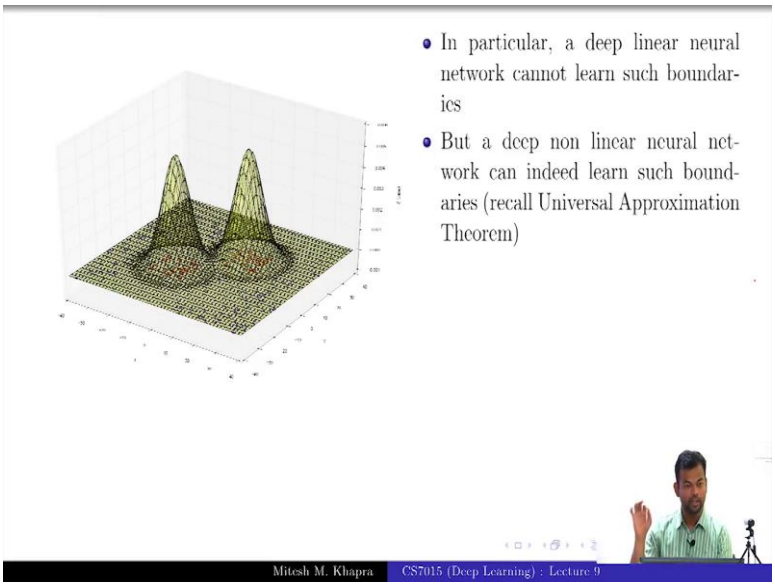
(Refer Slide Time: 01:53)



So, what you are learning eventually? We will just y as a linear function of x and initially at some point we started off with such linear functions right? w transpose x in the case of perceptron and MP neurons.

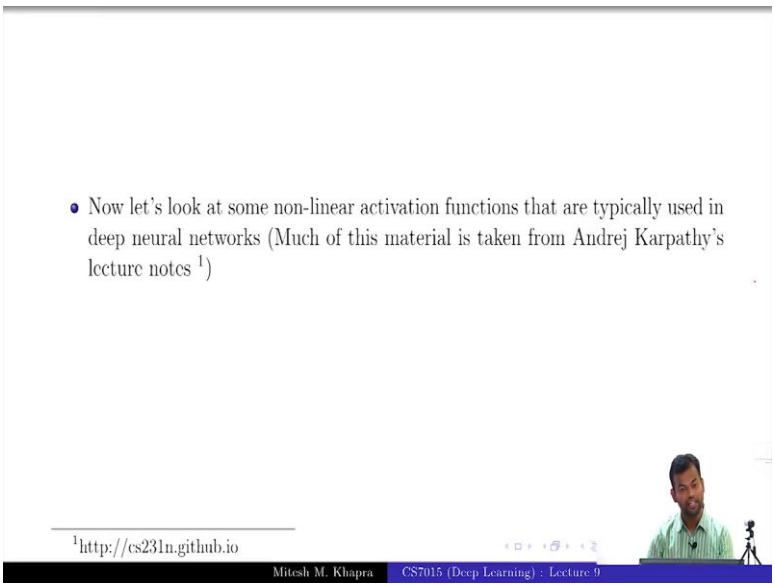
So, what does that lead to what kind of decision boundaries does that lead to linear decision boundary. So, if you do not have these nonlinearities we cannot have these arbitrary decision boundaries will only be left with linear decision boundaries.

(Refer Slide Time: 02:20)



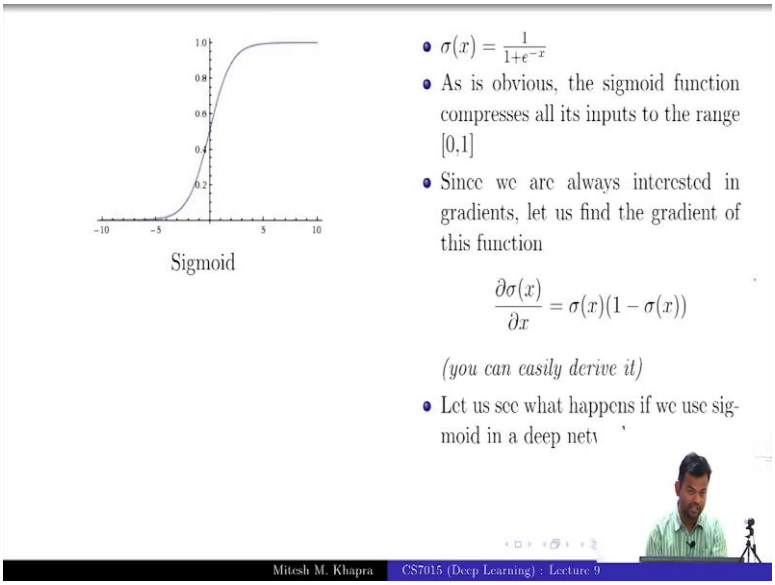
In particular, will not be able to solve this problem that we had right we were given some red and blue points and there was no way to draw a line such that the red points are separated from the blue points. What we needed is some kind of circles or ellipses to separate the red points from the blue points that cannot be done with linear decision boundaries. That can happen only, if you use a deep neural network with non-linear decision boundaries and we actually have a proof for that. What that proof the universal approximation theorem actually towards, right?

(Refer Slide Time: 02:51)



So, that is why nonlinearities or the activation functions clear a very important role in the success of deep neural networks, right? Hence you want to examine them very closely and see, what are the newer kinds of nonlinearities that have been proposed. So, we always start with the basics. So, will start with sigmoid see what are the problems with sigmoid? And then see what we can do to solve some of these problems?

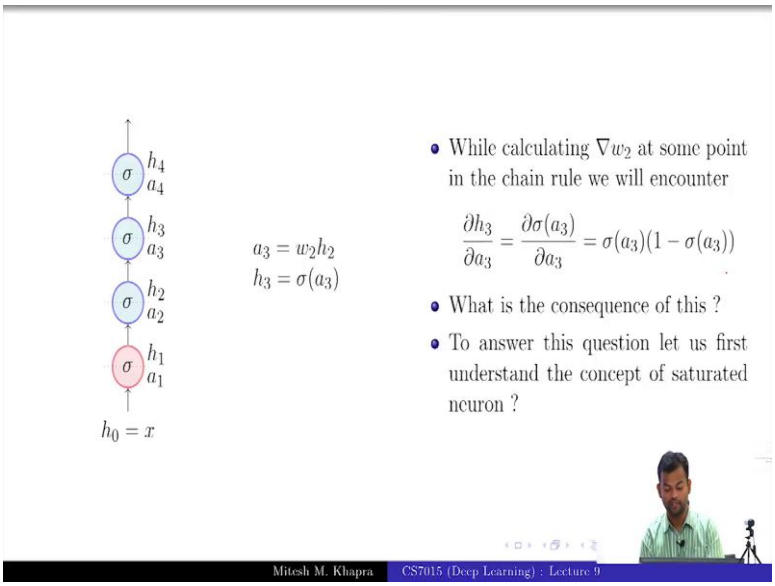
(Refer Slide Time: 03:13)



So, this is what the sigmoid function looks like you have seen it a million times and it actually constrains the input to 0 to 1 right? So, it takes some input and it constrains it 2 values between 0 to 1. Now since, we are always interested in gradients right? Because, the entire training and that is why I did that precursor in the first module the training always depends on gradients.

So, it is always important to look at what does the gradient look like? So, we know what the gradient looks like? We have computed this is just sigmoid of x into 1 minus sigmoid of x. So now let us see what happens if you use such a sigmoid neuron in a deep neural network.

(Refer Slide Time: 03:49)



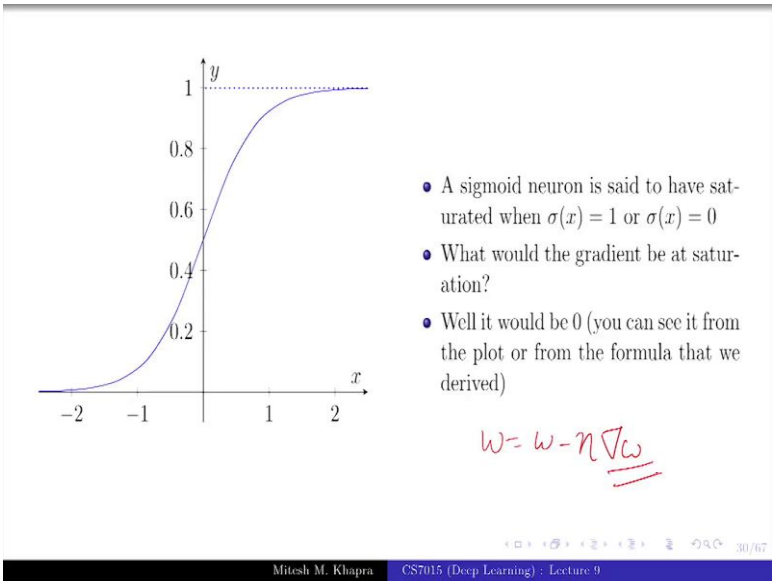
This is a deep neural network and without loss of generality I am going to use a thin deep network, but the same holds for a deep for a wide deep network also. So, suppose you are interested in computing the gradient with respect to w 2, right? At some point in the chain rule, you will have this term. How many of you are convinced about this? Ok. And that will lead to this. Could that cause a problem?

So, at some one of the terms in your chain rule is going to be this dou h 3 by dou a 3. I am assuming all of you are convinced about that and I have given you the exact formula for dou h 3 by dou a 3 will that lead to a problem?

Student: (Refer Time: 4:32).

Good. So, what is the consequence of this to answer this, we need to understand the concept of saturation, right?

(Refer Slide Time: 04:40)



So, a sigmoid neuron is said to have saturated. If it is output is 1 or 0 or rather close to 1 or close to 0 ok. What would happen in that case to the gradient?

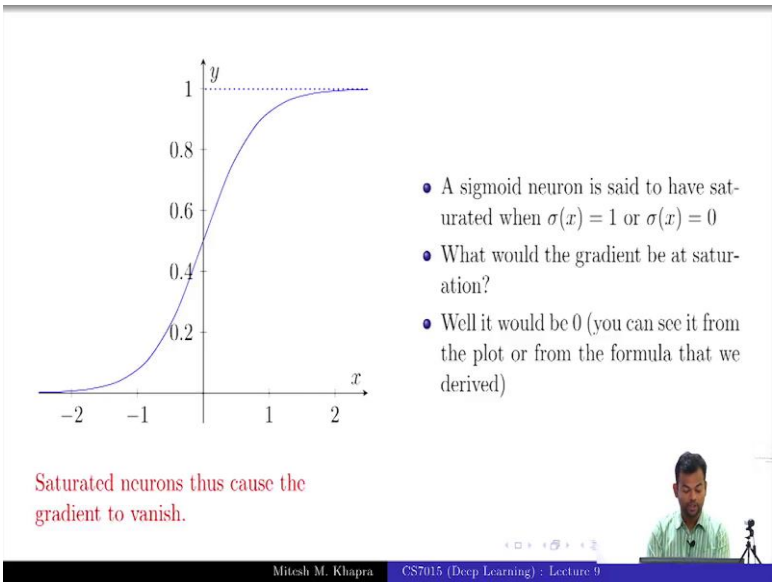
Student: (Refer Time: 4:48).

It will vanish right, because sigmoid of x into 1 minus sigmoid of x. So, it either extremes is going to vanish and you do not even need the formula for that. You can just see it from the diagram right. Because, the gradient here is going to be 0, that is obvious right. It just a what horizontal line.

So, this gradient would be 0. So fine, why does it bother us? What is our entire training premise based on gradients? Right, what does our update rule? What happens, if this guy is zero? No update e the weights just stay where they are right; that means, the training gets stalled right?

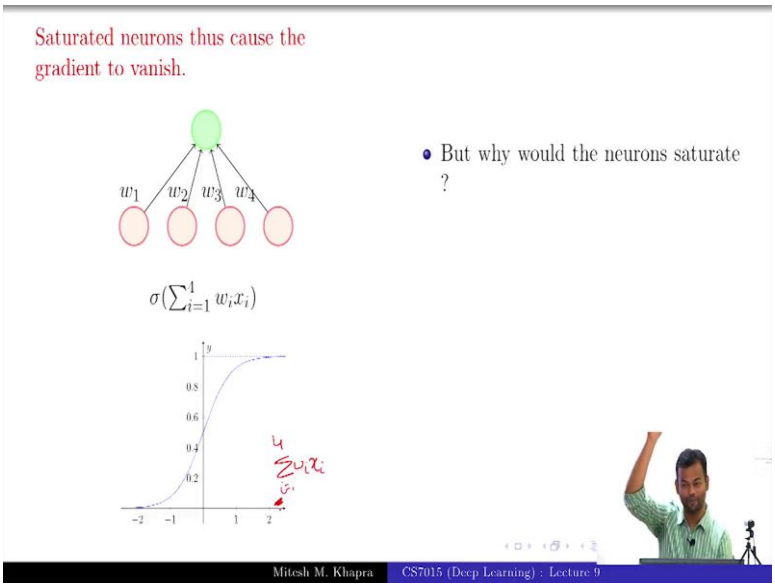
So, think about this right? If all the neurons in your network have saturated; that means, all the weights the gradients will be 0; that means, all the weights will remain where they are you pass another input. Nothing is going to change right. It still be 0 so, if this neurons have saturated your training will just stalled, ok? So, that was one of the reasons which is to cause problem in training deep neural networks earlier, right?

(Refer Slide Time: 06:05)



So that is one of the reason, why it was not converging? Because these weights used to these neurons is to saturate. So, this is one problem with sigmoid neurons a saturated sigmoid neuron can cause the gradient to vanish.

(Refer Slide Time: 06:15)



But, why would the neurons saturated? I mean, what would cause them to saturate? Ok, this saturate find their gradients will vanish, but why would they saturate? We should be able to get some hints from the figure that has been drawn. So, this is actually that x needs to be changed.

So, on the x axis we have x quite obviously, but that has to be something else. So, what it is? What is happening is? What does the sigmoid neuron do? It takes this aggregate it or someone just disappear (Refer Time: 7:26). So, is it very boring today no right? So, you have this aggregated sum of the inputs. Once, you have that aggregation you applied the sigmoid now tell me when would it saturated.

Student: (Refer Time: 7:39).

When the aggregation is very large; that means, one of the 2 things could happen either the x's are very large or the w's are very large. Would the x is x is be large I see a lot of you saying no why?

Student: (Refer Time: 7:45).

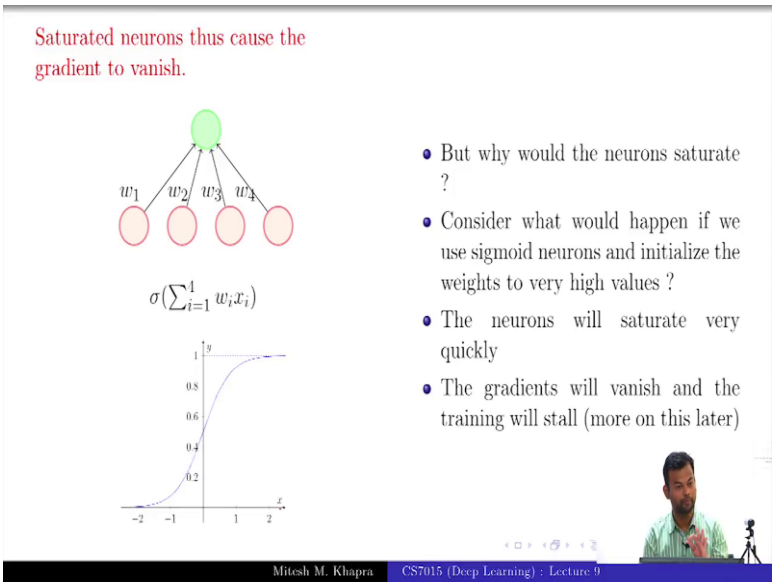
Good, we normalize them, right? We make sure they are between 0 to 1. So, we do not allow those arbitrary large values of pressure density and so on, right? We make sure they are between 0 to 1. So, then the weights can be a problem, right? Now, why would the weights be lies move later first?

Student: (Refer Time: 7:58).

If I initialize the weights to a large value, if I initialize all my weights in my infinite wisdom to a large value, what would happen right from the first training example itself. W i x i would take on a very large value and your neurons will start saturating. So, imagine if all the weights throughout my network are initialized to large values.

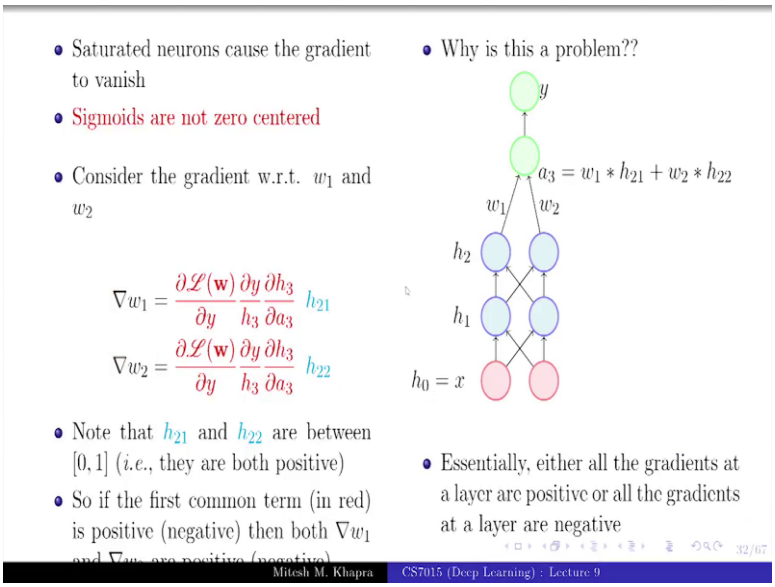
Then right from training instance 0, my neurons will start saturating and I will not be able to train anything. How many of you experienced this while doing back propagation? And the others did not do the assignment, they copied it please raise your hands. How many of you experienced it? Now many more hands will be raised still now ok. Honest people that is a paradox.

(Refer Slide Time: 08:20)



Consider what would happen if you use sigmoid neurons and initialize the weights to a very high value. They will start saturating and hence, you will have this problem of vanishing gradients, ok. Everyone gets this, so this is a problem at this sigmoid neurons.

(Refer Slide Time: 08:37)



The other problem with sigmoid neurons which is very interesting is that they are not 0 centered. What do I mean by that? They are not 0 center that is, what it ok. So, sigmoid is are not 0 centered. What do I mean with that? Mean by that they are not 0 centered;

the value is between 0 to 1 right? So, the average cannot be 0 it is always going to be above 0 ok. Sigmoid neurons are always going to take on positive values between 0 to 1.

So, why is that a problem. So, that is an interesting explanation. Oh did I say that did I put the acknowledgements somewhere. So, all of this material that I have been talking about it is taken from Andrej Karpathys lecture notes. So, here is this interesting explanation for this.

So now, consider this particular network ok and I am going to focus only on this part; that means, the output layer and just the layer before that and the layer before that has these 2 weights w1 and w 2. I am going to focus on that.

So, to update these weights, I need to compute? So, what do we need to compute gradient? Ok now, you will answer. So, we need to compute the gradient with respect to w1 and w 2 and this is what it is going to look like? What is the red part and blue part? Why red and blue the red part is dash for both common for both, right?

So, this is going to be common I do not know why I did that? Ok (Refer Time: 9:39) So, this red part is common for both and what is the blue part actually? What is dou a 3 by dou w 1 h 2 1 and dou a 3 by dou W 2?

So, dou a 3 by dou w 1 is just h 2 1 and dou a 3 by dou w 2 is just h 2 2 ok. So, let me just plug in those values and note that h 2 1 and h 3 are between 0 to 1. So, can you make some interesting commentary on this interesting, but useful not just philosophical stuff? That these 2 derivatives are for the weights at a given layer, I have just taken 2 weights, but I could have taken n weights and the same thing would have hold.

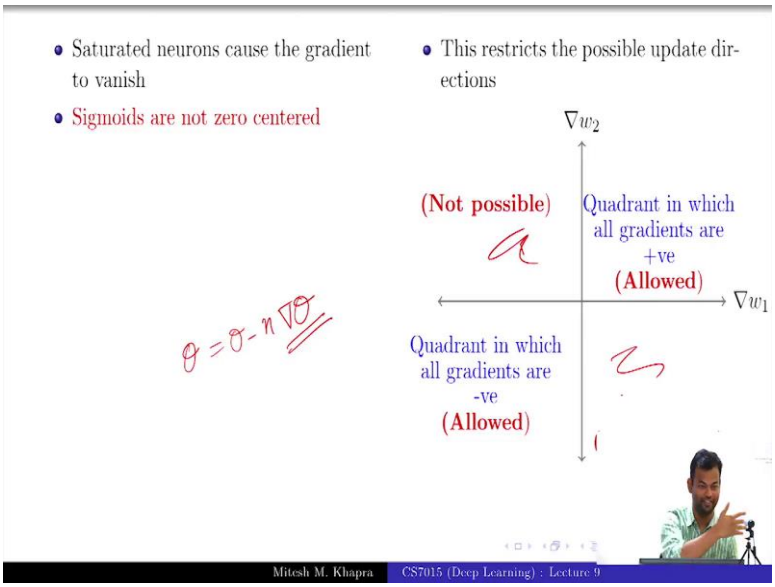
Because I know that the derivative is proportional to the input that it gets and the rest of the part is going to be constant because that is coming from the chain rule up to the previous layer, right?

So now, what is happening because of that, just to make fun of you guys. I mean, if you get that sorry, good yeah it is not very straightforward, but let us see. So, if the first common term in red is positive, right? Then, what would happen to these 2 guys? They would both be positive right, because h 2 1 and h 2 2 are positive.

Now, the first common term in red is negative then, what would happen to these 2 guys? Both negative so; that means, the gradients of the weights at a particular layer where, either all be positive or they will all be negative you get that, that is because of this. Common part and the blue part the blue part we know is positive.

So, what matters is the common part and that common part can either be positive or negative for all of them together right? That means, for a given layer all the gradients at a layer are either positive or they are all negative. So, let us see what is the implication of that right?

(Refer Slide Time: 12:14)



So, this actually restricts the possible update directions.

So, which is the quadrant? Which has all positive first? Ok sorry, for embarrassing yeah and all the negative is the third quadrant; that means, your movements can only happen in the first quadrant and the third quadrant. So, do you see a problem with this right? So, you are going to actually try to move that your theta.

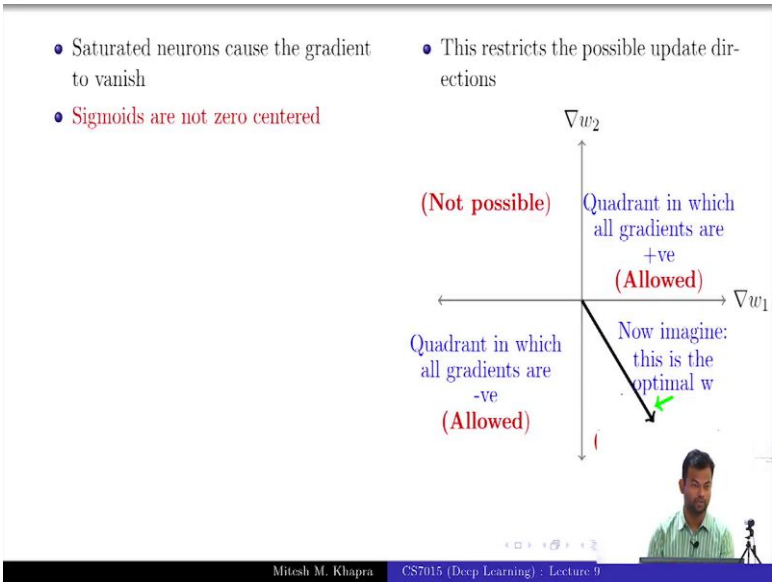
Which is a collection of w 1 and w 2 is theta minus eta into the gradient right. And you know that, this vector which is the gradient vector can either be positive; that means can lie in the first quadrant or it can lie in the third quadrant. These movements are not possible; that means, there are certain turns or certain movements or certain directions,

that I am not allowed to take. So, what would this mean it would take a dash time to converge.

Student: Longer time.

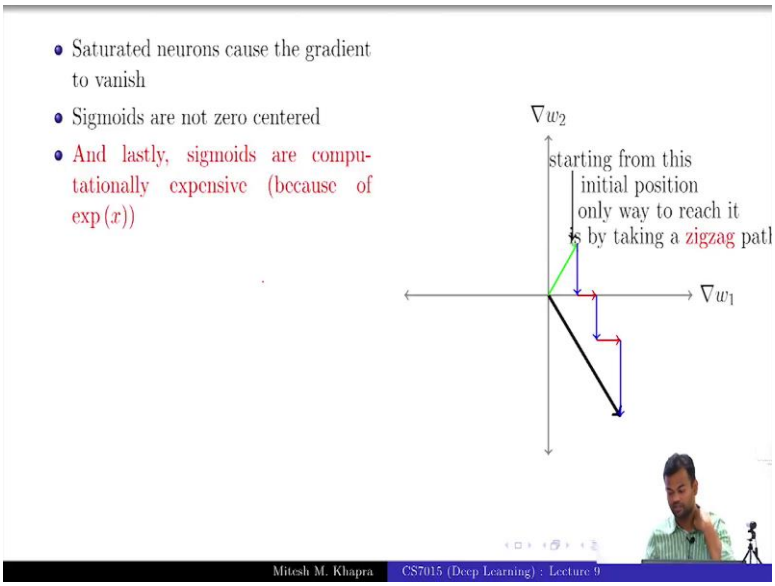
Longer time to converge right because, I am restricting my movement; so, imagine you have to go from destination to destination b and I say that you can never take a right turn right. And there is some going to be some problem, it will take longer to reach there unless the directions are to our left right unless your destination is (Refer Time: 13:22), but that will not happen.

(Refer Slide Time: 13:25)



So, suppose this is the optimal w star.

(Refer Slide Time: 13:29)



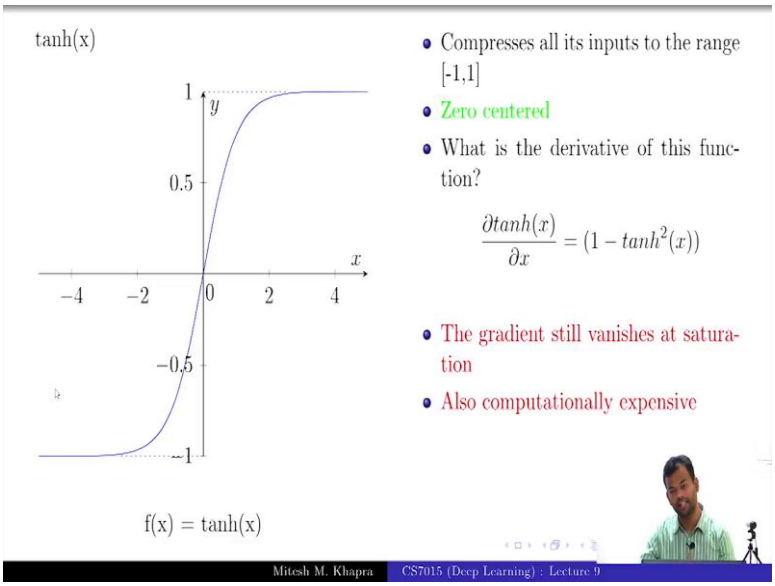
And we start with some random initialization because, that is why we are going to start. Then the only way I can reach it is, I may by making a series of this kind of movements right. As the exact pattern is, what will have to take? Because these are the only movements which are allowed or some movements which are allowed and it will lead to a certain cryptic pattern and I will not be able to have the complete freedom of moving in the direction. Which would have directly, taken me to the optimal.

So, that is a problem with something not being 0 center. And lastly sigmoids are expensive to compute because; you have to do this EXP right. It is not something as easier as something else that we will see in the lecture today ok. So, these are some problems with sigmoid functions.

Student: (Refer Time: 14:15).

So, this is some issues that were they with sigmoid functions. So, this pointed that ok, maybe we should try better activation functions.

(Refer Slide Time: 14:23)

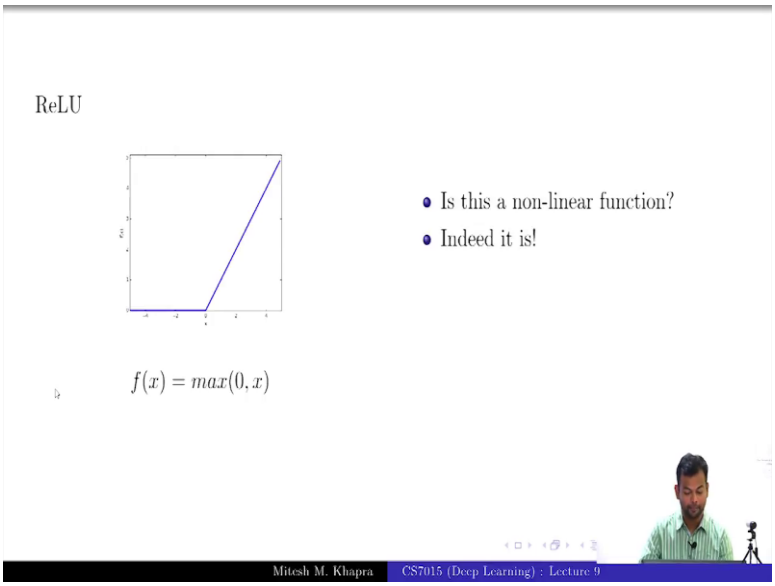


That is why tanh become very popular, but tanh is not something which happened post 2006 right. So, this was like 92 or 93. When I think Yan Lacunae had started moving to tanh from sigmoid functions right. Now again here other inputs are compressed between minus point to 1 ok. Where inputs are now 0 centered which takes care of this problem which I mentioned at the end.

That these directions of movements are constrained and was the derivative of this function 1 minus tanh square right? What happens at saturation even without looking at the formula? The gradient would vanish to 0 right? So, the vanishing gradient problem is still there.

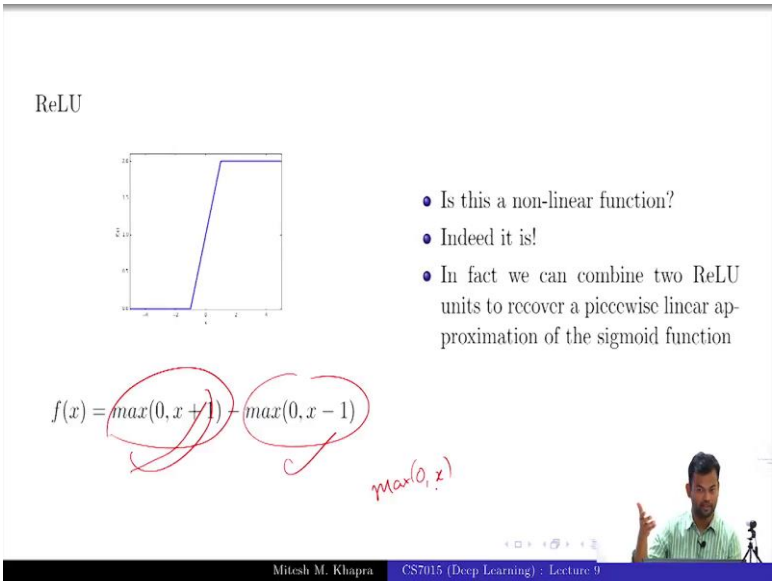
What you have solved is a problem of 0 centering? And that itself used to give better results than just using a sigmoid function, but it is still computationally expensive because, you still have to do these e raise 2 components right. The you still have to compute these exponential powers. So, it is still computationally expensive.

(Refer Slide Time: 15:17)



So, then in around 2012 I guess is when this ReLU was introduced in the context of convolutional neural networks, right? And this is what the ReLU function actually looks like. Is this a non-linear function? It just looks like a line right? Why is it a non-linear function? It is a non-linear function, right? Because, x is you cannot write x the output as a function of I mean as a linear transformation, right? So, you have this 0. In fact, if you take 2 ReLU functions smartly.

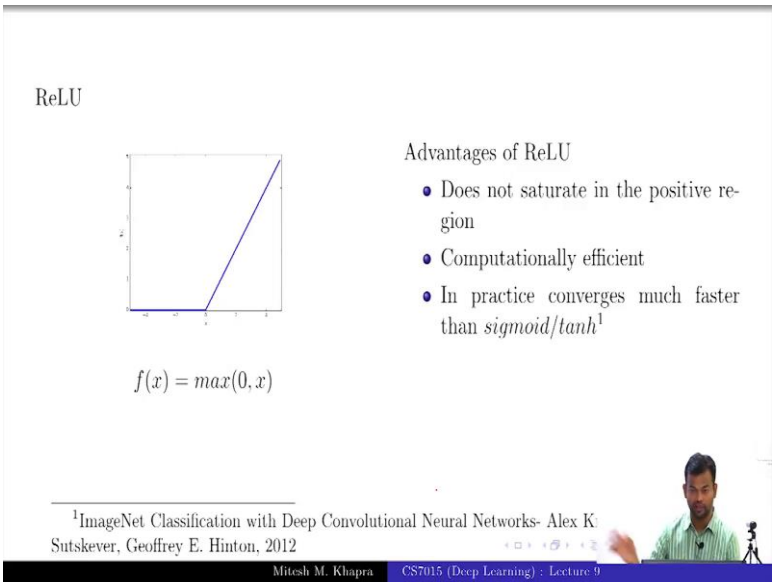
(Refer Slide Time: 15:50)



You can actually get the sigmoid I mean you can get an approximate for the sigmoid function. So, you can go back and check this ,right? So, if you take these 2 functions and subtract one from the other. What is this? This is a ReLU function this is also a ReLU function right?

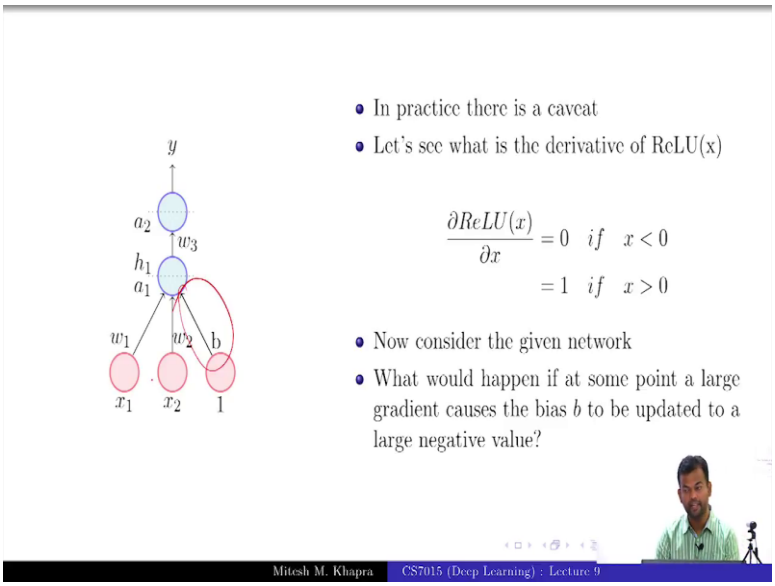
So, I define ReLU as max of 0 comma x. So, both of these are ReLU functions some variant of that and now, if you subtract one from the other. You will actually get a approximation of the sigmoid function right. And this cannot happen if you have 2 linear functions, take any 2 linear functions, you will not be able to get this kind of an approximation.

(Refer Slide Time: 16:27)



So, ReLU is a non-linear function. What are the advantages of ReLU? One is, it does not saturate in the positive region right? It is computationally very efficient the output is either 0 or x. There is no powers nothing like that right, and it practice it converges much faster than sigmoid and tanh. So, that is what this 2012 paper show. And now, ReLU has actually become more or less the standard in all Convolutional neural networks.

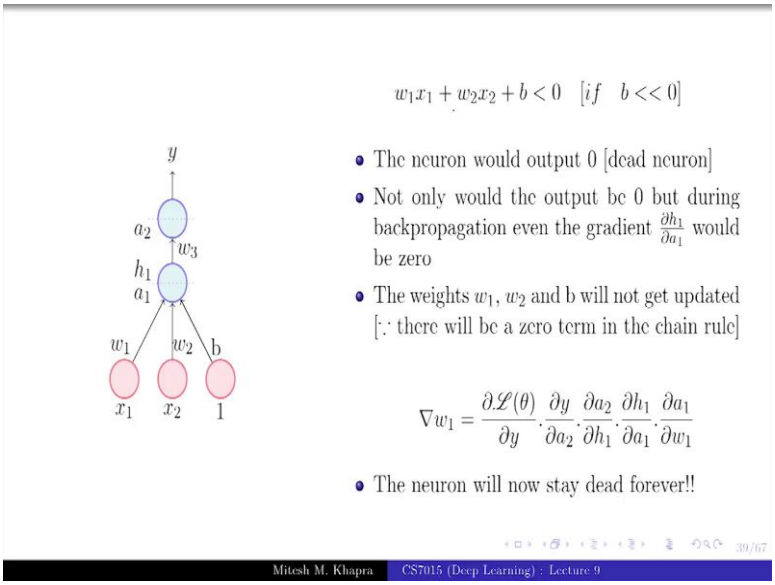
(Refer Slide Time: 16:53)



But there is still a caveat while using ReLU ok? So, the derivative of ReLU we can see that if x is less than 0. Then, the derivative is going to be 0, right? And if x is greater than 0, then the derivative is going to be 1 and that straight away follows from the definition of ReLU which is 0 or x.

So, when it is 0 the derivative will be 0 and when it is x the derivative will be 1. So now, consider this given network and let us assume and this is not a very far faced assumed. Assumption it can happen in practice that, at some point a large gradient causes the bias b to be updated to a large negative value. So, what I am saying is that something happens and b gets updated to a large negative value.

(Refer Slide Time: 17:41)



Now, what would happen to this quantity remember this quantity which I have circled is actually the input to the blue colored ReLU neuron that I have. So, I am asking you what would happen to that input? That input would become negative.

So, the neuron would output 0 and I am calling it a dead neuron. Why? If the input is 0, I mean is a input is negative then, the ReLU functions, output would be 0. What would happen to the gradients during back propagation 0? That means, what would happen to the weights?

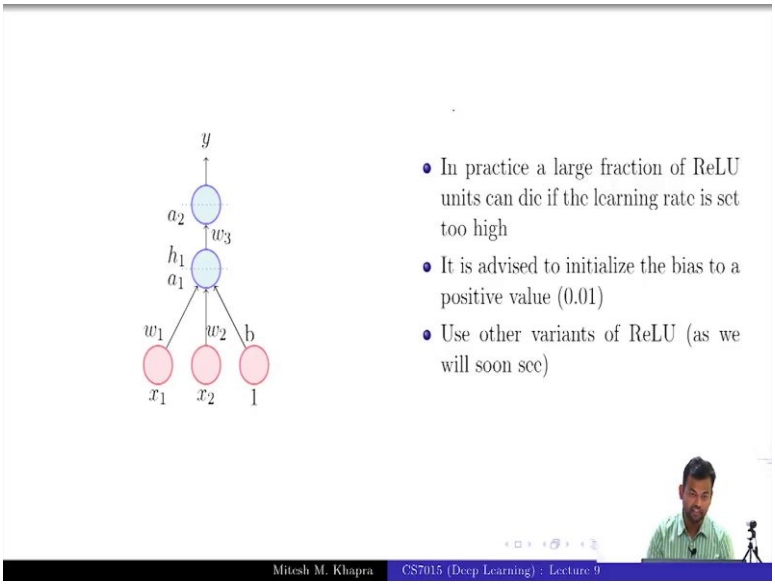
Student: (Refer Time: 18:06).

Would not be updated right now, but that is fine right. If you give some other input this will recover. Why am I calling a dead means permanent? Right, unless you are in some fantasy world, but dead is dead right? So, why am I saying that it is dead? I could might as well I would give it a next input and then probably things would be ok, bias is still very negative because nothing is getting updated right or bias is still very negative. You know that x 1 and x 2 are constrained because, you have normalized them right and w 1 and w 2 have not been updated.

So, still the situation does not change. So, what happens is that once a ReLU neuron dies because, somewhere in the chain rule you got a 0. It will stay dead forever ok; it will never be able to come out of that. It will always produce a negative output; that means,

that output will be clamped to 0; that means, no gradients will flow back and; that means, all the weights will not get updated connected that neuron.

(Refer Slide Time: 19:16)



So, in practice when you train a network with ReLU, you will observe that a large fraction of the units can die if the learning rate is set too high. why this if condition?

Student: (Refer Time: 19:30).

What was the assumption that I made, that the bias receives a large negative update. And that is possible if your learning rate is very high. Because, you got some small negative gradient, but your learning rate blew it up.

Now, what is the practical implication of this? If a training a network and a large number of your ReLU neurons have died. What does it mean? Most parts of your network are? dash useless; they are not learning any feature nothing right is all 0; that means, you have this large number of parameters versus getting wasted. Because, they feed into a ReLU you function and the ReLU function just keeps outputting 0.

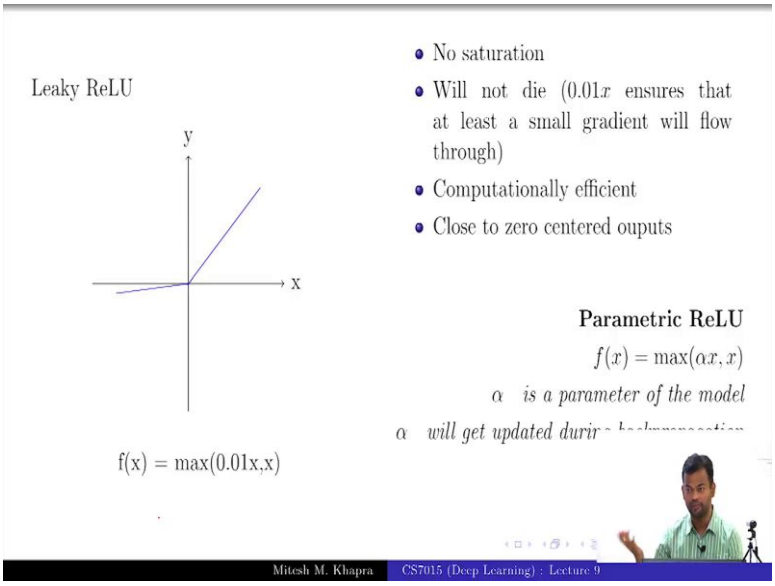
So, if you have n neurons in the particular layer and most of them are 0; that means, you are not really learning an n dimensional feature representation. You are just learning a much smaller feature representation, right? So, can you give me a simple way of one simple way of avoiding this among many other ways?

Student: (Refer Time: 21:07).

No dropout is statistical right, it is probabilistic this is like always dead one thing is to update the weight to a large to a positive value and 0.01 mind you is a large positive value right. Later on we will see y, but 0.01 is reasonably large ok. So, were going to initialize the bias to a positive value.

So, that even if this large negative gradient flows through there is still a chance that it will not become very negative. And hence, it will not mess up the things the way it does that is, one solution to that right, but still you will find that even after that the ReLU neuron a lot of those can die, but still in practice they work better for a deep convolutional neural network ok and we can also use other variants of ReLU.

(Refer Slide Time: 21:31)



So, there have been to avoid this dead neuron problem. There are other variants of ReLU, which have been proposed, and that is what we look at next. So, there is something known as a leaky ReLU. Is it obvious from the equation, what it does right? So, instead of producing 0 it will just produce a very small value proportional to the input. Now, what would happen to the gradients? They will not saturate right? Will have the gradient would be, if the input is negative what would the gradient be?

Student: (Refer Time: 21:56).

0.01 right so that means, some gradient will still flow through. How many if you get this? Right so; that means, if you use a leaky ReLU neuron some gradient would still flows through. So, just understand this trend right that ah. And this is I mean all this stuff is simple there is nothing great in this, but just put it in context right?

So, in 2006 to 2009 people realized. Ok now, we can trained networks and maybe whatever, we have done with unsupervised pre training actually corresponds to better initializations or better optimizations or better activations and so on.

So now let us try doing research in that. So, that led to the discovery of ReLU, now people started observing problems with ReLU and then proposed a variant of it which is leaky ReLU right? So, that is how this area has now become very prolific and grow right. So, we started off with this seed idea that, it is possible to train these deep neural networks and now we are trying to make arrive at better and better ways of doing it.

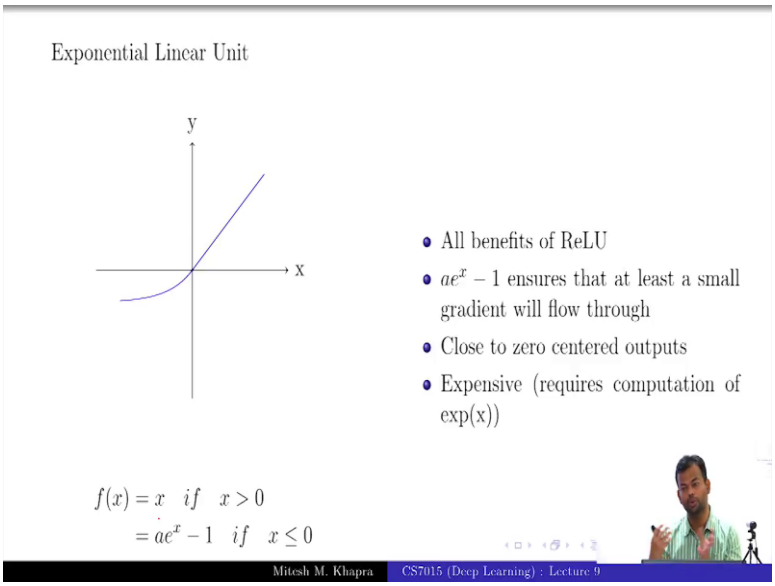
Making it more and more easier to train them and take care of some of these irregularities which existed earlier. So, one of them being sigmoid not being a very neat function to optimize with right? So, that is what all this is about. individually all of these are probably easy for you to understand. Once you go back and look at the slides you all this is nothing great in this.

But, what I want you to really understand is this bigger picture of what is happening here as long as you get that time frame with. And of course, leaky ReLU is again computationally very efficient. There is no exponents no squares nothing like that, and it is close to 0 centered and it is still not 0 centered, but close to 0 centers because, you have outputs on both side and then someone came up with a generalization of this, which is parametric ReLU. So, y 0.01 make it alpha x and alpha will also be a?

Student: Parameter.

Parameter, it is a trainable parameter it is not a hyper parameter. Ok, how many of you know the difference between parameter and hyper parameter? Ok, you have used this in the back propagation as I am right? So, it is a trainable hyper parameter it will get optimized along with your other parameters in the network.

(Refer Slide Time: 23:55)



So, then someone said leaky ReLU fine parametric ReLU is fine. Let us try to do exponential ReLU ok. So, it has all the benefits of ReLU it ensures that, at least a small gradient will flow through even when your inputs are negative; that means, it avoids this dead neuron problem again close to 0 centered outputs, but it is expensive because now we have added this exponential right?

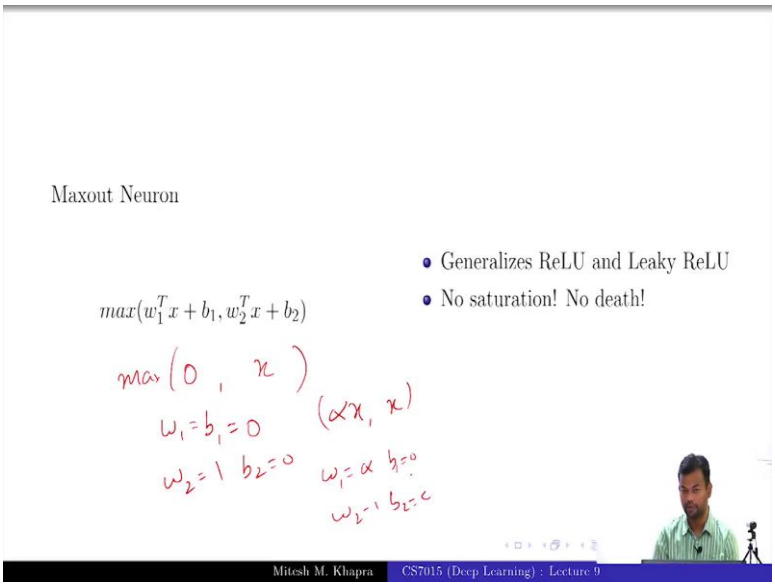
So, these are all ideas which came out during this period and all of them were shown to work better than the other and so on. And of course, at the end I have to tell you a final conclusion right. Whenever, I give you so many possibilities.

So, I have given you sigmoid, tanh, ReLU, parametric, leaky, exponential. Now, what do you use? Right this the idea is not to confuse you, but to give you one solution, which would largely work yeah? What regularization?

Student: (Refer Time: 24:45).

Yeah, you could have done yeah that (Refer Time: 24:48) there is exactly. So, a lot of this research right, which has happened in this period. It is not a lot of it is juristic right; you solve one problem with ReLU. Ok, the neurons and saturated ok just make it something which does not saturated.

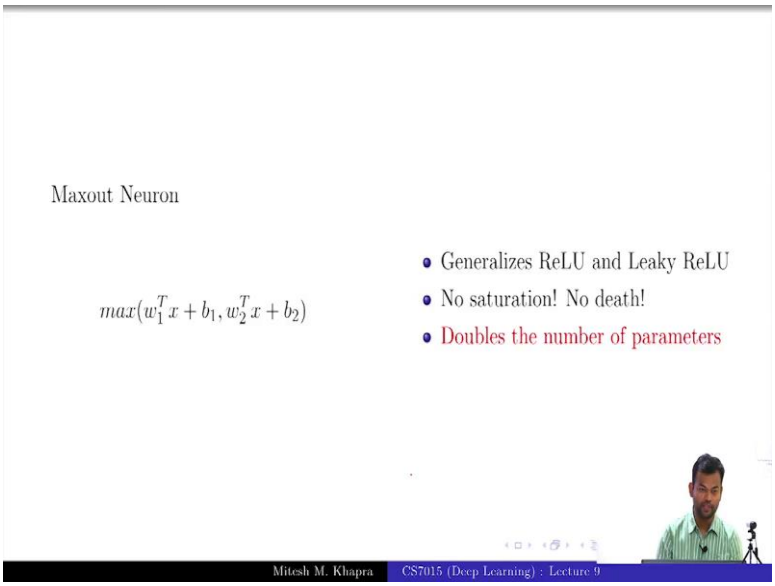
(Refer Slide Time: 25:05)



So that is there, it is possible that the other solutions would also go there is not that. This is the only solution which works now, then someone came out with max out neuron which is a generalization of ReLU and leaky ReLU. Why do I say it is a generalization? What was ReLU? That means, w 1 equal to b 1 equal to 0, w 2 equal to 1.

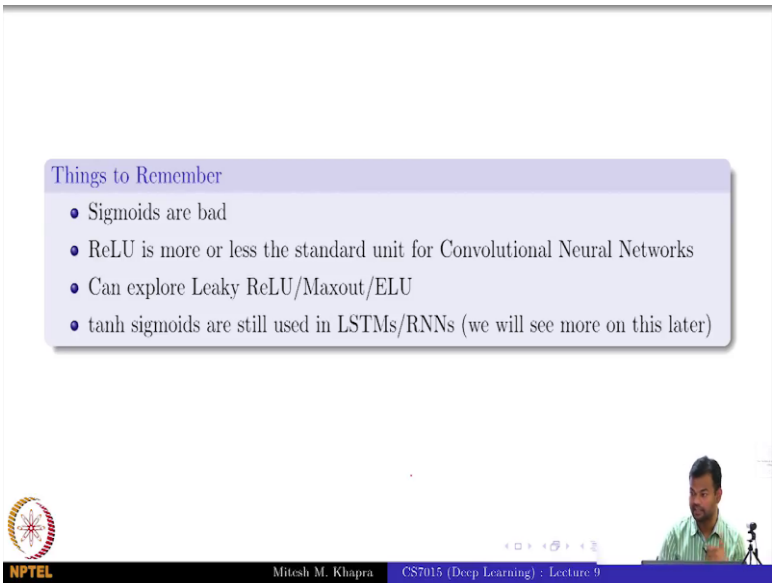
So, it is a special case of the max out neuron. What about leaky ReLU?, this was parametric value, but again what about. So now, what is happening w 1 equal to alpha b 1 equal to 0, W 2 equal to 1 b 2 equal to 0. So you see how it generalizes right. So, this is how these variants keep, kept coming up

(Refer Slide Time: 26:05)



Now, the problem of course is, doubles the number of parameters right because you earlier had only w transpose x plus b. Now, you have w 1 transpose b 1, w 2 transpose b 2 and so on right. So, it is actually doubling the number of parameters that you have.

(Refer Slide Time: 26:18)



So now coming to the final conclusion of all of this right? What you need to remember is that sigmoids are bad.

So, no one uses sigmoids in convolutional neural networks. They still use somewhere, I am I am sorry, about this ReLU is more or less the standard unit for convolutional neural networks.

So, any standard CNN that you will pick up it will use ReLU as the activation function. If you want you can explore leaky ReLU max out ELU and so on, but it will require a lot of careful tuning. Say, if you want to use something out of the bulk box ReLU is just fine ReLU just works fine in practice despite all this dead neuron and other problems.

Student: (Refer Time: 27:20).

Yeah. So then, the argument for that is that, how often when you reach the point x equal to 0 right? So, the chance of that having is happening is very, very low. And if you get there, you can always approximate it by some epsilon or something and for that training instance just go on right? Any ways you are making so many approximations with stochastic and mini batch and so on.

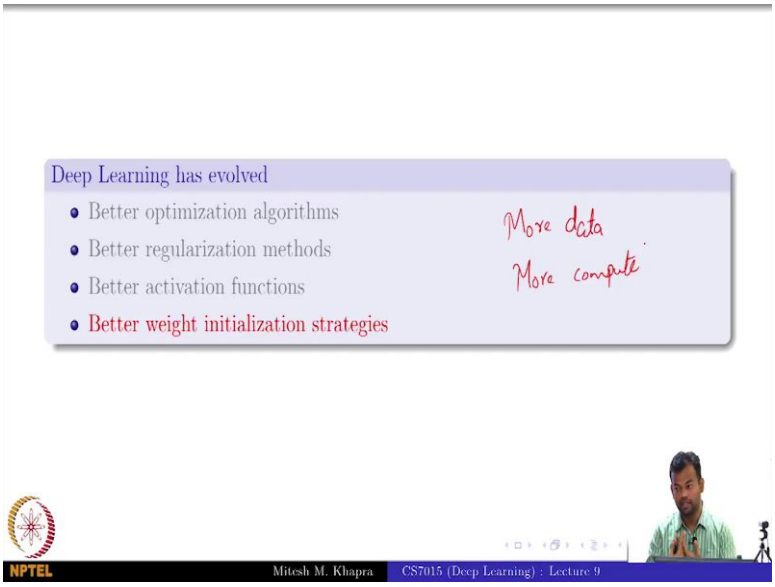
So, this is one more approximation that is how people typically deal with it, but in most cases, it will not come in that point appearing is very low, but the question is valid and tanh sigmoids are still used in LSTM's and RNN's, which you will see at some later point in the course ok. So, there are a couple of more modules that I need to do. So, we just take a break here.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 9.4
Lecture – 09
Better Initialization Strategies

So, in this module we will talk about Better Initialization Strategies.

(Refer Slide Time: 00:17)



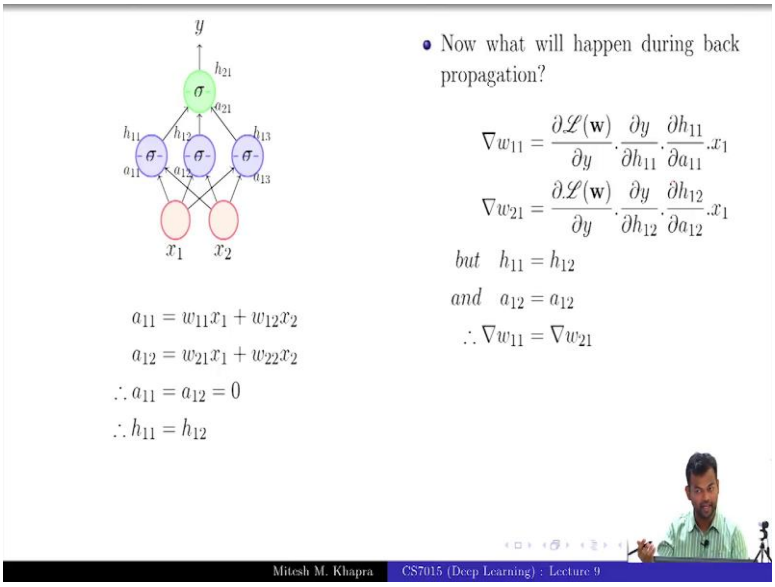
So, this is where we are in the story right. We saw that deep learning has evolved and at least these are the 4 things which have happened. So, we have by the way this slide is incomplete. What are the other things which have happened actually which you already said in the beginning 2 more things which are not technical, but which happened more data right?

And more compute, but these are not really technical in the sense that, I mean this just happened we have large amounts of data; that means, more data means, what if you are more data for training. You would have complex networks, but not over fit right because, you have so many, so much of data right? And more compute of course, it speeds up some of these matrix computations which happen.

So, remember in a deep neural network most of the things which are doing are matrix, matrix operations right? You are taking are that is what exactly you did in your back propagation assignment. You did a lot of matrix vector computations and so on, and the advent of GPU's this became, very very fast rate orders of magnitude fast.

So, this 2 which are here as nothing much to talk about that is just something we all understand, what has happened they. And so now, we will talk about better weight initialization strategies.

(Refer Slide Time: 01:27)



So, let us start with this question right. We will take this network and we will ask this question what happens if we initialize all the weights to 0. I like it when you all try to visualize it and ok. So, you have to see what happens right. So, let us start with a 1 1, which is w 1 1 x 1 plus w 1 2 x 2.

So, I always start small l. I do not try to see what will happen everywhere, just start with one neuron and see what happens take a 1 2. What would a one month's value b? If all the weights are initialized to 0 0 and a 1 2 again 0 right and same for a 1 3 it is all the way all the neurons in this layer are going to be 0, till is it.

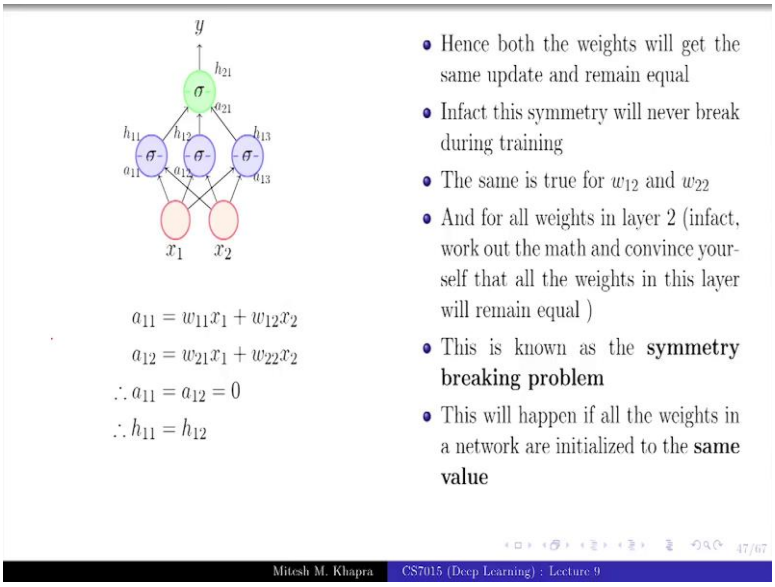
So; that means, they will all get the same activation. So, if the a's are same the h s are also going to be same and that is obvious irrespective of what non-linearity you use now

what will happen during back propagation what will delta w 1 1 b. This again I do not know why you do this ok, anyway that will be erase it, into x 1.

So, remember that the gradient is always proportional to the input. And you have somewhere along the lines along the chain rule. You have this h 1 1 and a 1 1. Just remember that and what would delta at gradient of w 2 1 b is that fine.

Now, can you see some things on the left hand side and make some comments on the gradients. We have seen that a 1 1 is same as a 1 2 and h 1 1 is same as h 1 2; that means, these gradients are going to be equal right; that means, the weight started off at the same value. They are going to get the same updates and again remain at the same or different value, but this same right. Then of course, move from where you started will not be 0 anymore, but they will all be at the same value.

(Refer Slide Time: 03:24)



Both the weights will get updated with the same value and they will remain equal so, but fine as I keep training they will move away from each other right. This is what I told you, when you feed in the first example take a both the weights remain the same, but now if you feed another example and you keep feeding batches there. There is no dearth of data that, you have and eventually these weights will move away from each other.

The update is the same again. The weights are the same again the same situation will hold right. Again your w 1 1 x 1 plus w 1 2 x 2 is going to be the same as w 2 and x 1 plus w 2 2 x 2 and the same argument repeats.

How many if you get this, ok. So, once you initialize the weights to 0 in all subsequent iterations, the weights are going to remain the same. I mean, they will move away from 0, but they will all be equal ok and this symmetry will never break during training. So, what actually is happening in terms of the capacity of the network this is same as?

Student: Single line tying the weights.

The same as tying the weights so, this symmetry will never break during training. So, asking what is the net effect which is happening?

So, you have so many weights in your layer, but all of them are moving together will so in essence. You do not have the same freedom as you have with n different weights right. Here, in some sense unintentionally tied them because, it started off with the same value. Now you are all moving at the same values rate you are all going to the same value. So, you do not really have the amount of freedom that you would actually expect with n different parameters all of you get this.

And the same is true for w 1 2 and w 2 2 also which are the weights connected to the second neuron and this is. In fact, true for all the weights in layer 2 you can actually mathematically verify it; that means, whatever this small analysis that I did here. Just go back and do it for all the weights in the network and you will see that all of them. If you are going to initialize them to 0 all of them are going to remain equal.

This is known as this symmetry breaking problem this is are known problem. This is existed much before 2006 and so on. If we initialize all the weights to 0 you will have the symmetry breaking problem. Is there anything sacrosanct about 0 or would this happen even if you initializer to same, but non 0 values and that should have been cleared from the iteration right. Because, after the first iteration we were at non 0 weights and after that the story repeated, right?

So, even if you initialize it to non zero weights the same story is going to report repeat so; that means, as long as you initialize all the weights to the same value. You are going

to end up with this symmetry breaking problem ok, which is not good. So, what is it that we have learnt about initializing weights?

Student: (Refer Time: 6:32).

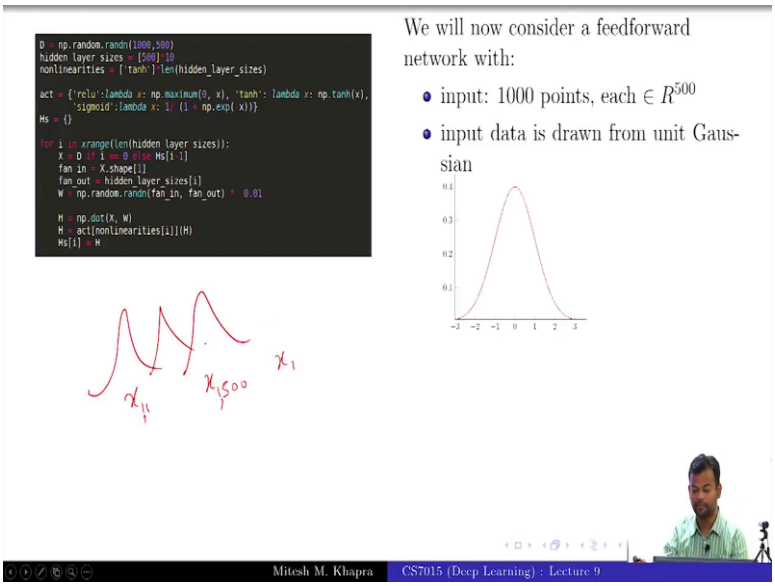
Definitely do not initialize all weights to 0, definitely do not realize them to the same value ok. This is the first thing that you have learned. So, we are seeing different ways of not making the light bulb and then, we will come to a way of making it, So, 0 and equalist no bad yes, some weights will not get updates in that case right.

So, then that that should be fine so, that is the other thing I wanted to make at some point right, these 4 things right initialization, optimization, regularization and activation function. These are not independent things they are all tied to each other.

So, as you said now if you use regularization then probably you could be a bit careless with the initialization. Even if you had initialize the weights together drop out would have ensured that, some of these weights are not active at a particular training instance; that means, they will not get weight updates; that means, they will move away from the other weights.

So, that is this is not that only one of these things can be done right? You are going to use a combination of these things, but while analyzing them, we will just look at one of these things. Assuming that, the others are not being, right? So, will assume that we are not using drop order anything is that fine.

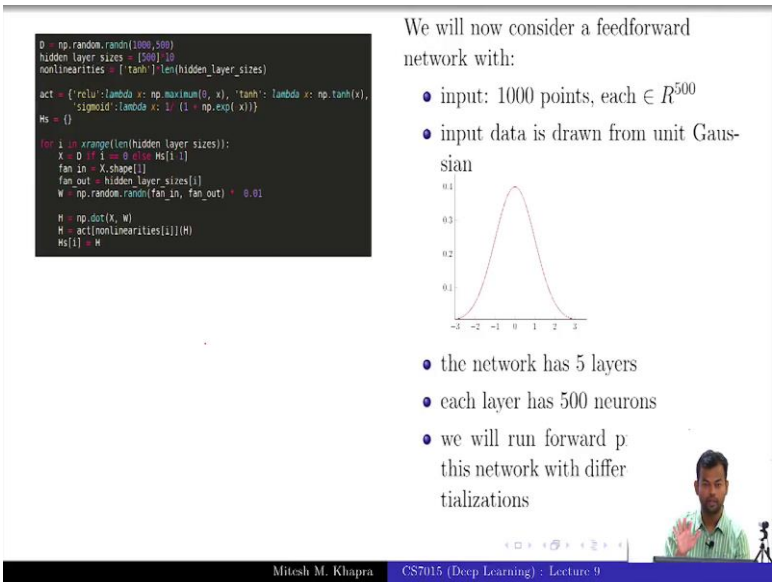
(Refer Slide Time: 07:01)



So, this at least this in practice you are not supposed to initialize the weights to 0s and equal values that is what we have learned so far. Now for the rest of the to convince you about some other weight initialization methods. What I am going to do is? I am going to take a feed forward network, where you have as input some thousand points? Each of this point is 500 dimensional. And the input data is drawn from a unit Gaussian. What I mean by that is? You have this x 1 2 x 1 500 rate for the data instance one.

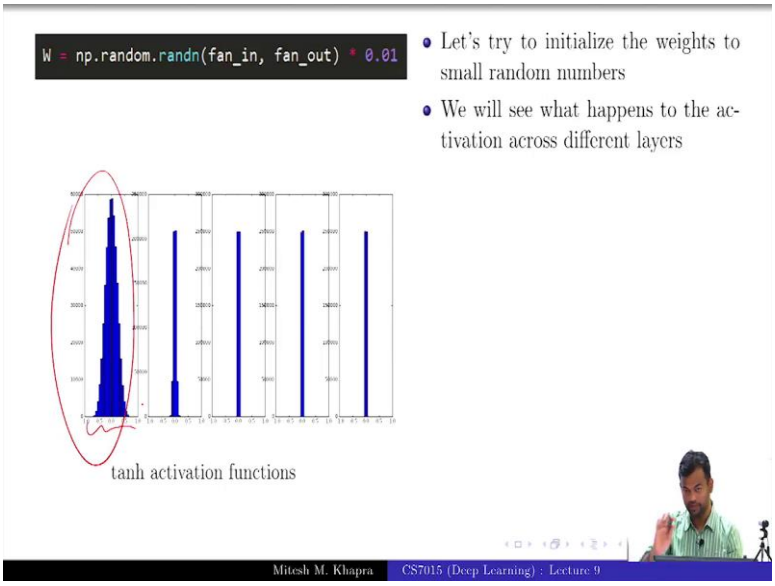
So, all of these 500 dimensions come from a unit Gaussian is that fine. So, this comes from a unit Gaussian. This comes from a unit Gaussian and so on ok. That is what I am going to assume?

(Refer Slide Time: 07:50)



And the network has 5 layers each layer has 500 neurons. The input is 500 neurons each of the 5 layers is also 500 neurons. And now, we will run forward propagation no backward propagation, no loss nothing and I am not even giving you an objective. This is just some input and I just want to see, what happens up to the last layer? I am not even bothered about the actual last layer; that means, I am not trying to minimize any cost entropy squared error loss anything.

(Refer Slide Time: 08:18)



So, let us try a few initialization strategies. So, we realize 0 is not good realize equal is not good. So, let us try some random initializations, but small weights ok and this is my way of randomly initializing with small weights.

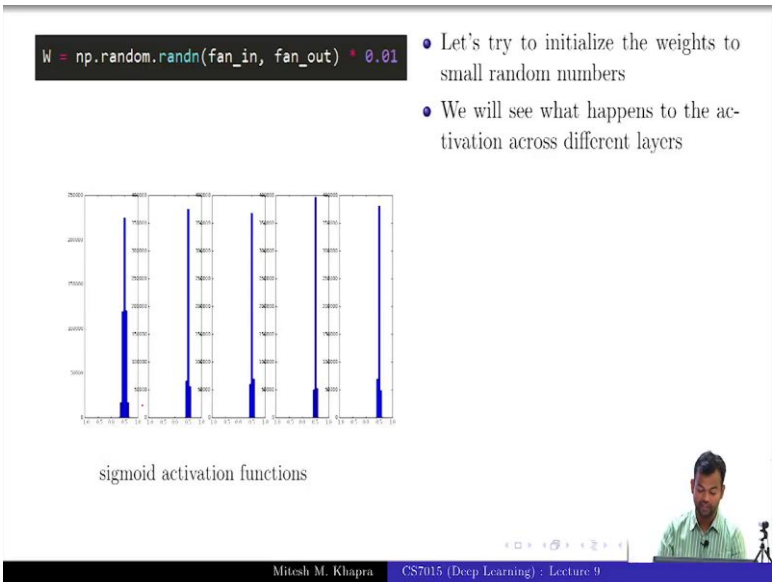
So, my W is a matrix of size fan in into fan out rate, which is n cross n ok the number of weights coming in and out rate. So, n cross n and I am drawing from a uniform distribution and then, multiplying it by point 0 1 which ensures that, all the weights are very small you get the setup now with this, I am going to start with the input and then keep doing these transformations.

So, I will do W transpose X plus b pass it through a sigmoid and do this 5 times. Because, I have 5 deep layers now, this is what happens to the activations across the 5 layers? So, the first layer remember that we had drawn from a unit Gaussian right. So, that is what the data input data looks like. So, this is the first layer which is the input data basically and then this is what happens across the different layers.

So, what is actually happening and this is for the tanh activation function. There is no variance in the output of. So, this tells me, so this basically tells me that, for all the neurons what is the average value that, I am getting right and I should ideally get some histogram that for some neurons I am getting the value minus 1 for some neurons minus 0.9, 0.8 and so on, but what this is telling me is as, I keep progressing across the layers all the neurons have very similar values and they are all close to 0.

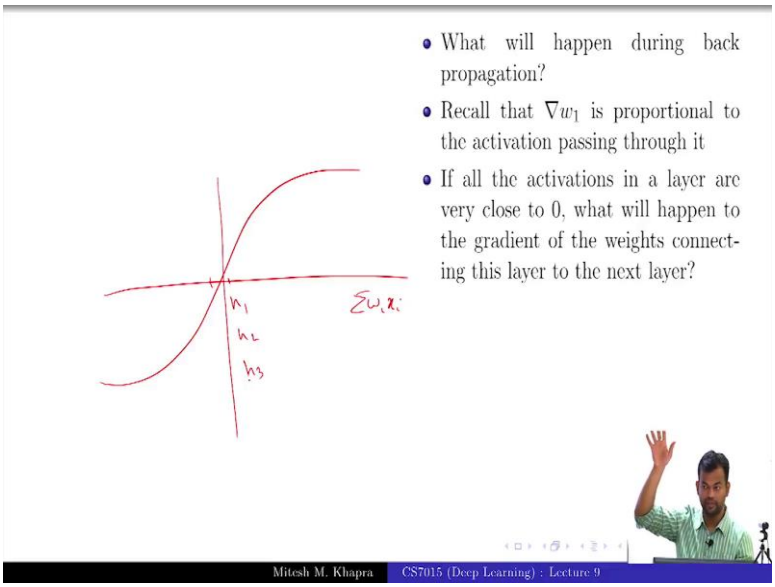
This is what actually happens in practice; I have just actually run it and computed the histogram.

(Refer Slide Time: 10:02)



And if I use sigmoid activation functions, again something similar all the values tend to be close to the center which is 0.5. So, this is 0.5 and although I had started with a nice Gaussian distribution.

(Refer Slide Time: 10:15)



Now, what will happen during back propagation? So, do not try to think for now that why, this happens I am just telling you have actually run the code. And this is what happens? Now given that, this has happened. What will happen during back propagation? So, all the activations in a layer are very close to 0, all the gradients are

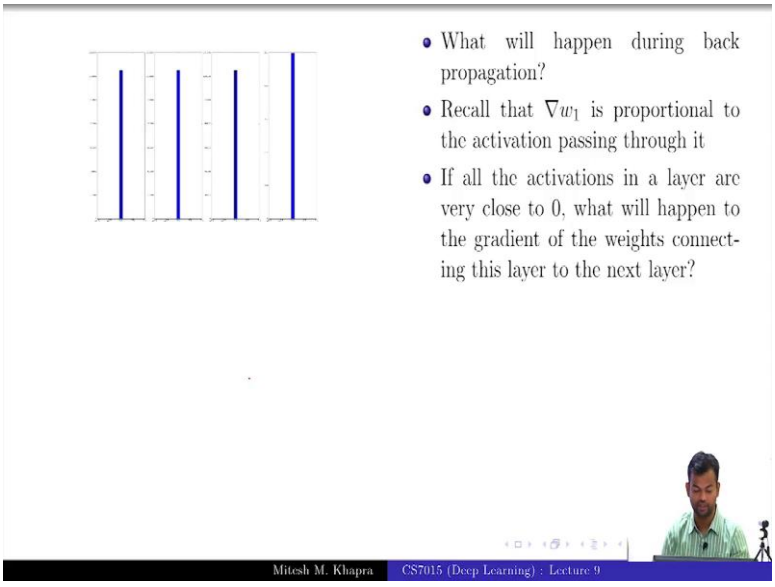
going to be close to 0; that means, no gradients are going to flow back; that means, which problem are we dealing with vanishing gradient problem.

So, if you initialize your weights to very small values and this is easy to see in the case of tan h. So, for tan h this is my function right and this is 0. Now remember that, this is wi summation wi xi, if all my weights are close to 0 or very small values. What is summation w ix i going to be it is going to lie somewhere here, right?

So, all these inputs are actually going to be very close to 0. Now if my inputs are going to be close to 0, I know that during back propagation at some point my gradient is proportional to the input, that, I have given and when I say input here I mean layer 1, layer 2, layer 3 and so on.

So; that means, all my inputs are very close to 0, now my gradients are actually proportional to the input. So, all my gradients are also going to be close to 0; that means, my gradients are vanishing right because, remember that across 5 layers you will have these products of gradients right. All of them are very close to 0. So, you will end up with something very close to 0 raise to 5. How many if you get this? Right, so our gradients are going to vanish.

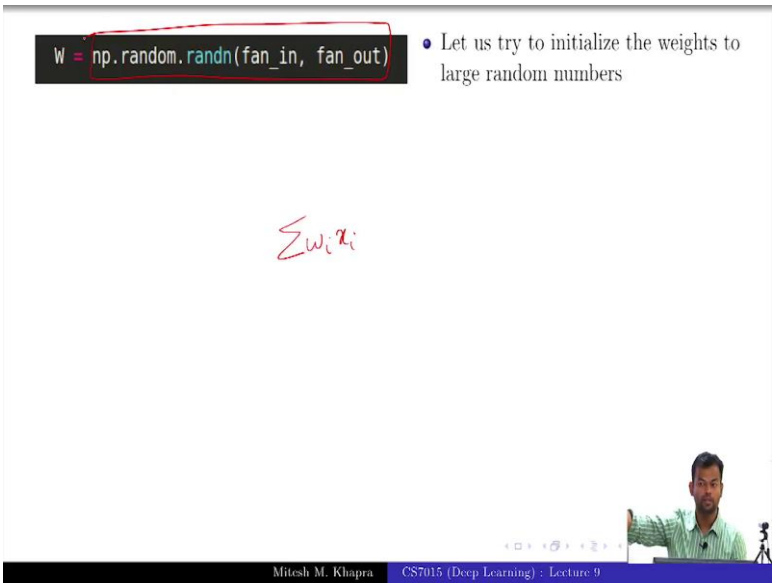
(Refer Slide Time: 12:06)



If you do this very small initialization of the weights and that is exactly what is happening? So, this is the histogram for the gradients and I see that, all my gradients are

actually very close to 0; that means, no effective training is happening. My weights are not receiving any updates this is what happens in practice. If you initialize your weights to very small values.

(Refer Slide Time: 12:25)

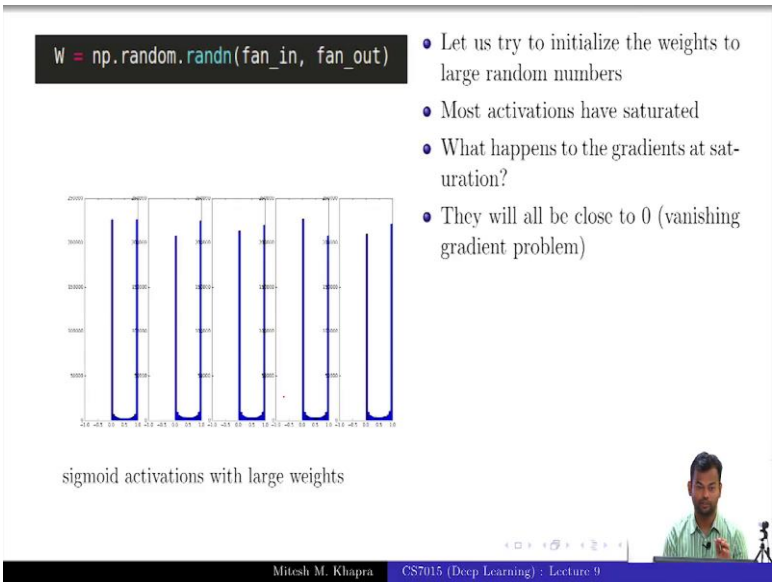


Now, let us try to do the opposite of this very small values did not work. So, let me try large values and for large values I just sample from the uniform distribution. I will get some numbers between 0 to 1. Now can you guess what will happen? Remember summation wi xi all your weights are large. So, why am I saying that number between 0 to 1 is actually large? It is not by all practical because; this is going to give me this function is actually going to give me numbers between 0 to 1. Why am I calling them large weights?

Student: (Refer Time: 12:54).

No, I will I just talked about the weights, assume there is no biases, how many of you get that answer? Remember there are 500 neurons. So, if you have 500 small values, that summation is going to be still large right. If you all of these are 0.4 or 0.5. Which still looks small, but if you have 250 of these or if, you have 500 of these the resultant sum could be somewhere of the order of 250 right. And that is very large because, if you pass that to a sigma and neuron, what will happen saturation right. So, you get this why I am calling these weights as large?

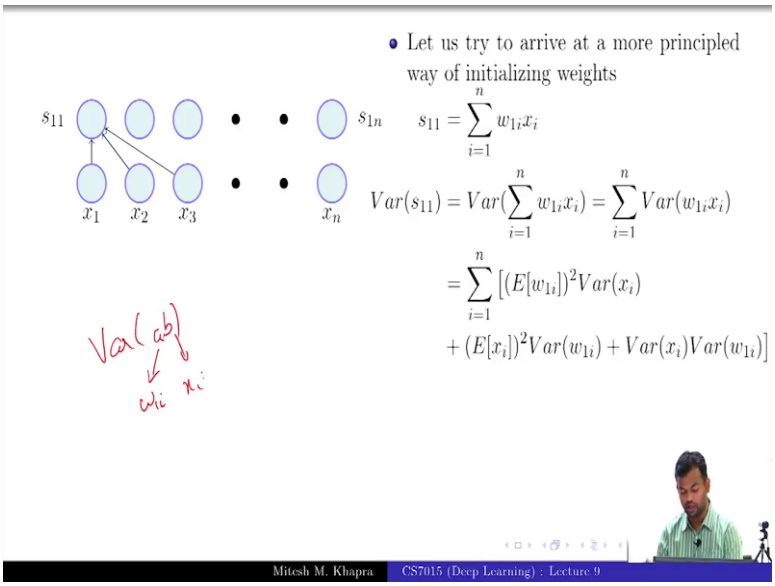
(Refer Slide Time: 13:30)



So, and this is actually what happens. So, when I have these tan h activations across all the 5 layers. I observe that my neurons saturate I either get minus 1 as the output or plus 1 as the output. And same thing happens, if I use sigmoid activations I either get 0 as the output or I get 1 as the output right. Neurons saturated means, what will happen gradients will vanish, right?

So, even if you initialize the weights to very large values all your gradients are going to be close to 0 because, they are going to vanish and again you have a problem. So, what have we seen so far? 0 is not good, equal is not good, small weights is not good, large weight is not good. Then, what do we do?

(Refer Slide Time: 14:15)



So, let us see what to do? So, let us try to arrive at a more principled way of initializing weights and this again do not should the messenger. I am going to give you a proof under certain assumptions ok. So, just bear with me, I just tell you what those assumptions are going to be as we go along. So, as I said right?

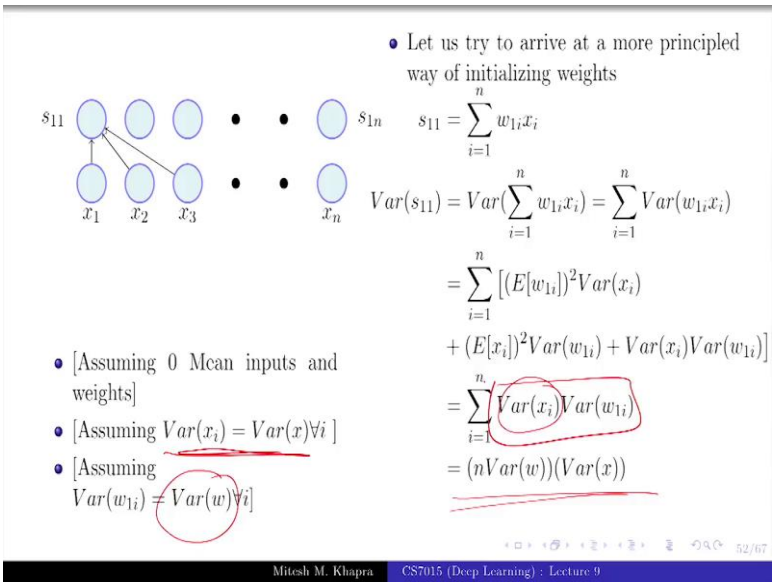
So, I mean you would argue that in practice these assumptions do not hold true, but at least they give us some insights into, what is happening, right? What is the overall idea behind? What is being proposed? So, let us start with that. So now, consider this deep neural network and I am just considering the first layer of it. Where I have this neuron s 1 1 and I am talking about things before the activation.

So, I know that S11 is equal to this quantity, right. So, all the incoming weights to the first neuron which is w 1 i into x i. Now for some reason, I am not telling you why I am interested in the variance of this. Can you tell me why I am interested in the variance? What did you see in the previous examples? There was no variance right, there was hardly any variance. So, let us see what happens if you compute the variance of this.

So, I am just taking the variance formula a variance of a sum is equal to the sum of the variances right. This is of the form variance of a into b, where a is w 1 i and b is x i. What is the formula for this? Or if you know it or do not know it do not care. So, this is the formula.

So, this is the generic formula for variance of a into b. Where you have to assume that a is w1i and b is xi. So, this is just a formula, there is no trick. Here, no math I mean, no nothing fancy here just apply the formula for variance of a b and substitute a is equal to w 1 i and b is equal to x i.

(Refer Slide Time: 16:07)

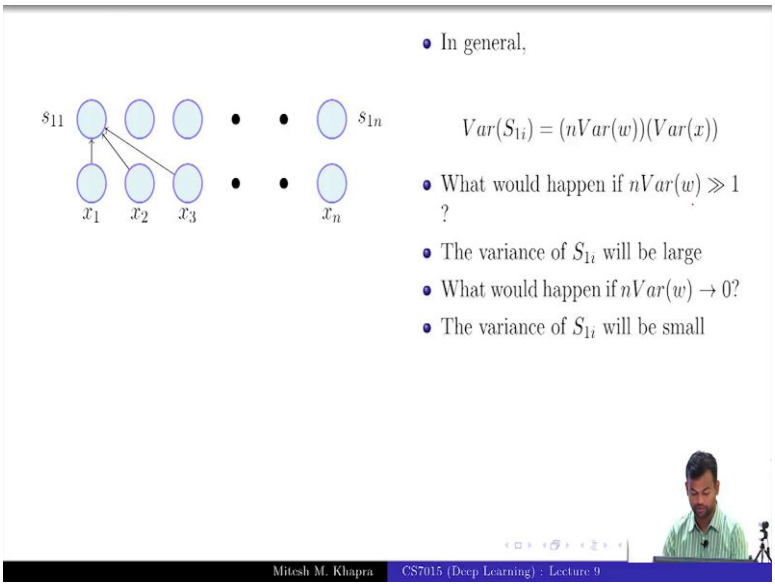


Now, I will assume that all my inputs are 0 mean fine. We have been assuming that, forever and all my weights are also from 0 mean ok. What is the effect of that which quantities will disappear? This will disappear because mean as 0 means the expected value of the weight is 0. So, the square of that is 0 an expected value of the input is 0 the square of that is 0.

So, what am I left with summation? Where I variance of xi into variance of w1i? Ok now I am going to assume that the variance of xi is equal to the variance of x; that means, it is the same for all the i's. So, I had this remember I had these 500 inputs.

So, I am assuming that for all the inputs the variance is the same. They all come from a similar variance distribution. And I am also going to make the same assumption for the weights fine and then I end up with this neat formula that the variance of S11 is equal to n times the variance of w into variance of x right? Because, I assumed that all these terms are equal and there are n such terms. Everyone is fine with the Maths so far with the assumptions that we have.

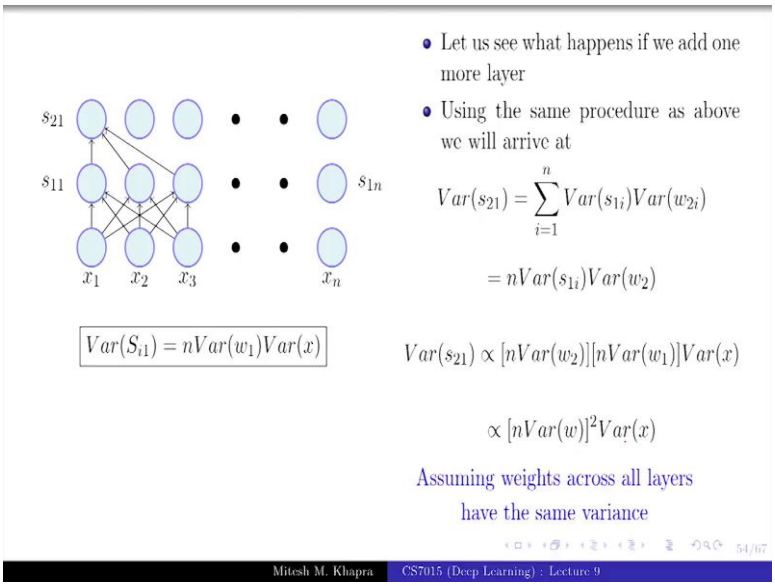
(Refer Slide Time: 17:17)



So, in general for any of these neurons right instead of just S11, I could take any S1i and this is what the variance is going to be variance would turn out to be? Because? I have assumed that all the weights and all the inputs come out from the same variance distribution ok from a distribution having the same variance.

Now, let us what would happen if this quantity is very greater than 1? The variance of S1i would be? Very large, right? And what would happen if this variance tends to 0 variance would be very low. So, I am just giving you 2 extremes to build the intuition and let us see what we are going to do with that intuition fine.

(Refer Slide Time: 18:01)



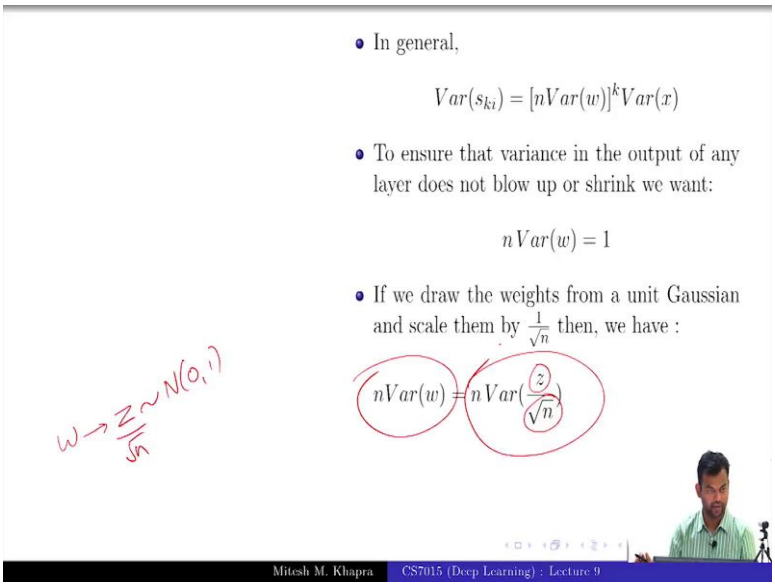
Now, let me add one more layer and see. So, I have added one more layer and using the same procedure as above. He will arrive at variance of S21 is actually given by this formula and actually what has happened here is, that this is si had xi earlier, but now instead of xi, I have S1i because those are the inputs to this layer, right?

So, this is exactly the formula that we had arrived at earlier. Assuming, 0 mean and the same variance for all the weights and the inputs and I am arriving in the same formula for the next layer where, instead of x i have S1i.

So, this will result in this quantity ok, but I already had a formula for a variance of S1i. What was that n into this quantity? So, I will just substituted it there. So, I can say that variance of S2i is actually equal to this. I just substituted this value.

So, that turns out to be, I have a square here when I have 2 here. So, you see where I am headed with this? What will be the variance of s k i. This raised to k and is on everyone gets this ok. I can just continue the same analysis and, I have assumed that these weights and always are the same variance, right?

(Refer Slide Time: 19:29)



So, in general I can say this ok. Now can you tell me something about when would this variance vanish when n variance of w is.

Student: Less than 1.

Less than one ok, and which is the thing that we should aim for you would want this quantity to be equal to one. In which case it will neither blow up nor shrink fine. So, so it to ensure that the variance is the output of any layer does not blow up or shrink. We should ensure that n into variance of w is equal to 1 right.. So, what is this, this just take a minute to understand this. I am saying that, I am going to initialize my weights.

So, I should initialize them in such a way that, the weights are coming from some distribution like we saw that the distribution was a uniform distribution from where, I was drawing the weights.

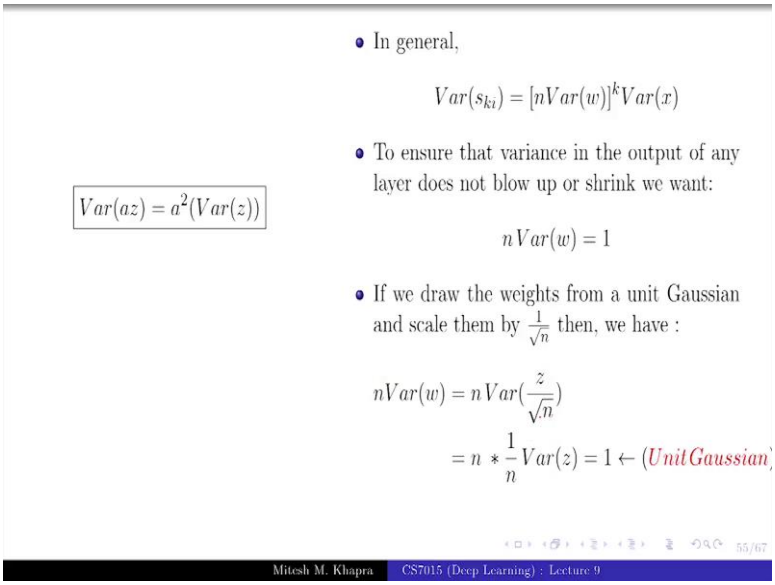
So, they are coming from some distribution I should try to draw them from a distribution such that, this condition holds. If this condition holds then across layers, my activations will not blow up or shrink though that is exactly. What was happening in the earlier case when I was doing those bad initializations with small values and large values.

So, let us see how to do that. So, what I am going to do is? I am going to consider a random variable z ok. Where is z comes from a normal distribution ok and I am going to scale it is value, I will draw from there and then I am going to scale it by 1 by square root

of n. What is n number? neurons in each layer right Here it is the same across all layers, but it could also be different. So, I am considering a particular layer and n is the number of neurons in that layer.

And now if w is actually equal to z by square root of n then, I can write that n into variance of w is actually equal to this quantity. Everyone is fine with this? There is no trickery here, I am just saying that, why I am doing this is not clear that will become clear, but at least what I am doing is clear? I am drawing the weights I am taking a random variable z which comes from a normal distribution and then, I am setting my weights to whatever values I have drawn I just divide them by the square root of n.

(Refer Slide Time: 21:49)



Now, let us see, what is variance of a z a square into variance of z? Hey that is a basic formula all of us know this. So now, what is variance of z by one in z into 1 by square root of n 1 by n into variance of z right. So, the n and n cancel and what is variance of z. What did I assume about z it came from a normal distribution 0 mean and unit variance?

So, variance of z is 1; that means, this quantity n variance of w is going to be 1. If I have initialized my weights such that, they are equal to this. right and now, do you see whether the weights are very small very large or what are they some now they made the weights dependent on the number of neurons.

So, if you have very large number of neurons, you are drawing drawing weights such that or you are initializing weights such that, it is some normal variable divided by the square root of n right. So now, when you do this summation w I x I your summation cannot blow up because, you have already divided it by n.

How many if you get this? So, this is a standard way used for initializing weights. How many if you tried this for your back propagation assignment? Why did you try this ah?

Student: (Refer Time: 23:12).

Because, you are having some problems with saturation I guess, right? So, this is how you should initialize your weights. This is more or less the standard technique and some variant of this right? Because, instead of n you would have this fan in and fan in out it, how many weights are coming in? And how many weights are going out?

So, you make it proportional to the square root of n into k or something like that right. So, but in general this idea right of course, this proof we arrived at it with lot of assumptions, but we at least got to some principle way of initializing weights and this is a largely used standard this and some variants of it.

(Refer Slide Time: 23:43)

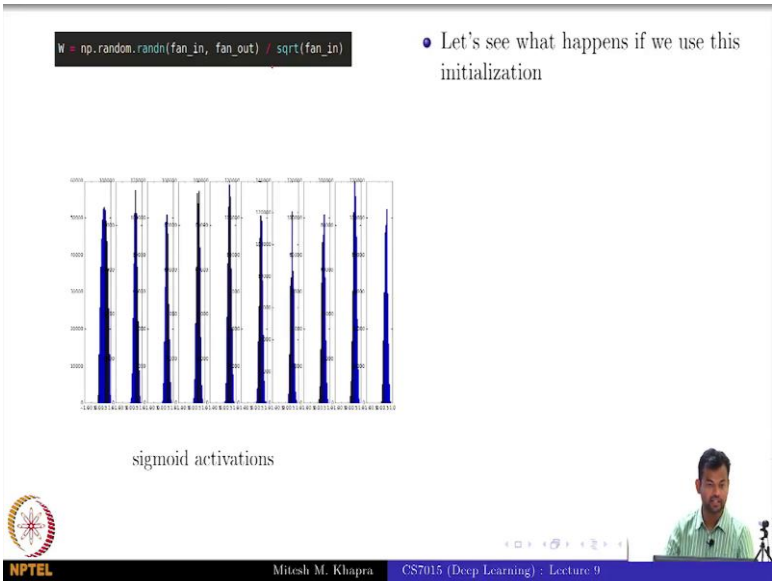


So now, let us see if I actually take the same network; that means 5 layers 500 neurons at every layer and then initialize it using this. So, this exactly what I had told you right that take it from a unit distribution sorry a normal distribution? And then, divided by the

square root of the number of neurons in that layer and now let us see what happens across the 5 layers.

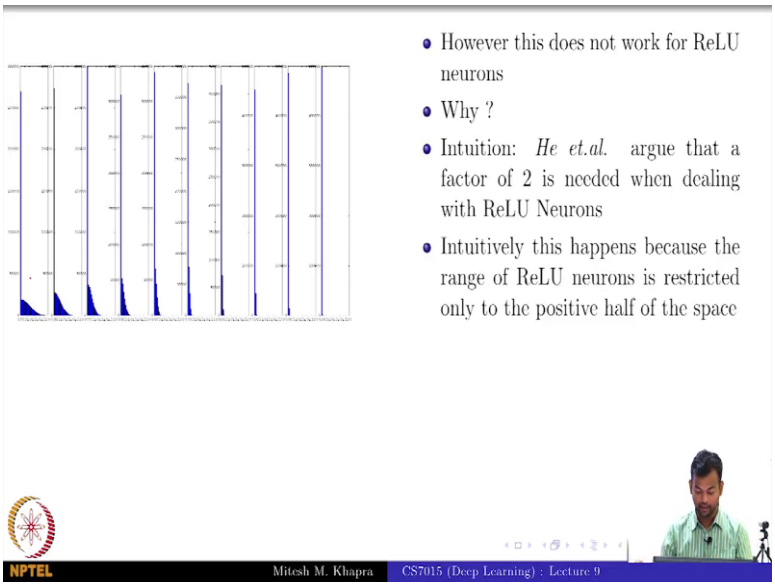
You see what happens we get this good variance in the activation functions. They are not all going to 0 or 1 or point 5, right? So, this solves the purpose for tan h activation and also for the sigmoid activations.

(Refer Slide Time: 24:18)



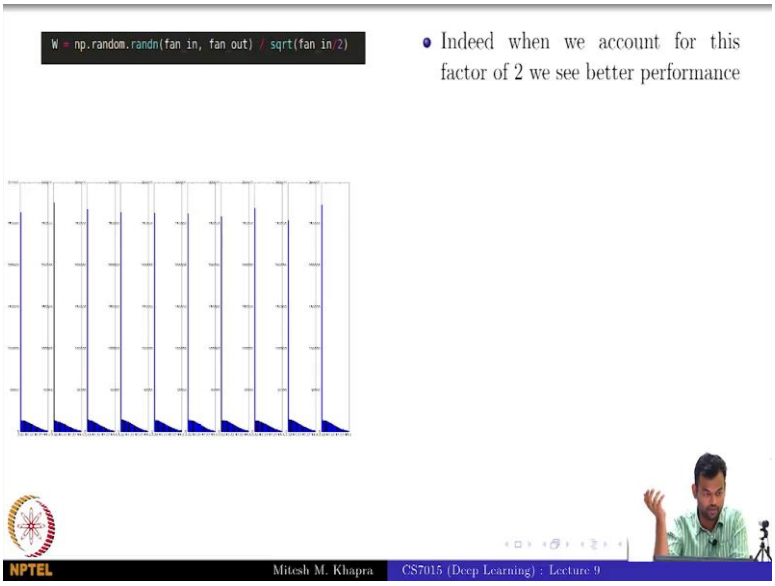
You see a good spread in the weights, and remember actually for sigmoid. Although, these values look close to each other, but this is the 0 to 1 range. This is actually minus 1 to 0 which will not happen for sigmoid. So, within the 0 to 1 range you get a good spread if you initialize the weights this way.

(Refer Slide Time: 24:35)



But it turns out that, this initialization does not work for the ReLU function. In the ReLU function you still see this effect that, you started off with a good spread, but as you keep going across depths this spread disappears, why would that happen to someone gave an intuition for this and is again one of those heuristic things. That in the case of ReLU you need to account for this divided by half. Because, half of the ReLU is not active right half of the ReLU is 0.

(Refer Slide Time: 25:10)



So, you need to account for that fact and do this simple trick that, instead of taking the square root of the fan in you. Take the square root of fan in by 2 because, you know that half the times it is not going to produce any output, right? So, that is a very simple heuristic that someone tried. And that leads to better activation functions better activations across all these layers, right? So, as you see across all the layers the spread is good now so, the same idea ok. So now, you have a good way of initializing neurons.

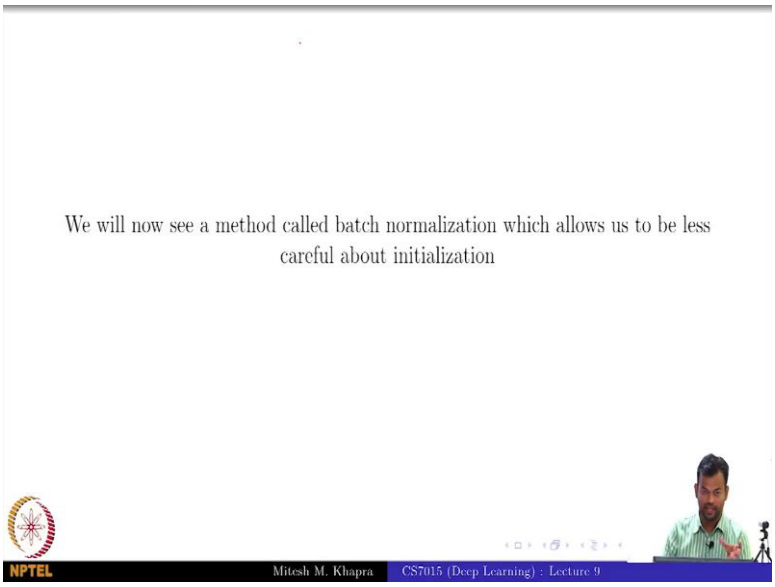
So, this should help you in your future assignments fine. So, this is how, what you have learned about? How to initialize your weights? And it makes a lot of difference to, how your network will behave, right? And that is what the I was trying to show that by computing these activations across different layers? And I showed that as you change these initializations strategies you get better activations.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology Madras

Module – 9.5
Lecture – 09
Batch Normalization

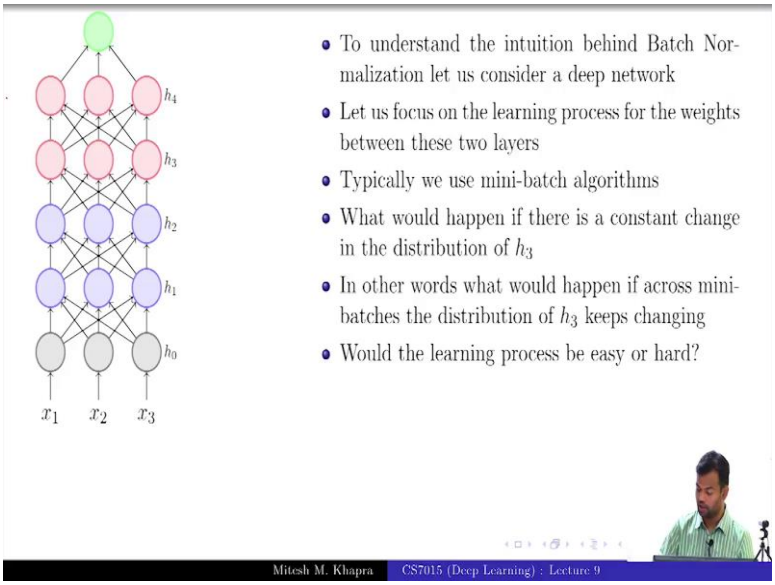
Now, we will end with something known as Batch Normalization, which is again almost a defacto standard at least in convolutional neural networks. So, if you are dealing with convolutional neural networks you will use something known as batch normalization.

(Refer Slide Time: 00:26)



So, let us see what it is, so this is again something which is some method which allows us to be less careful about initialization, so let us see why that happens.

(Refer Slide Time: 00:36)

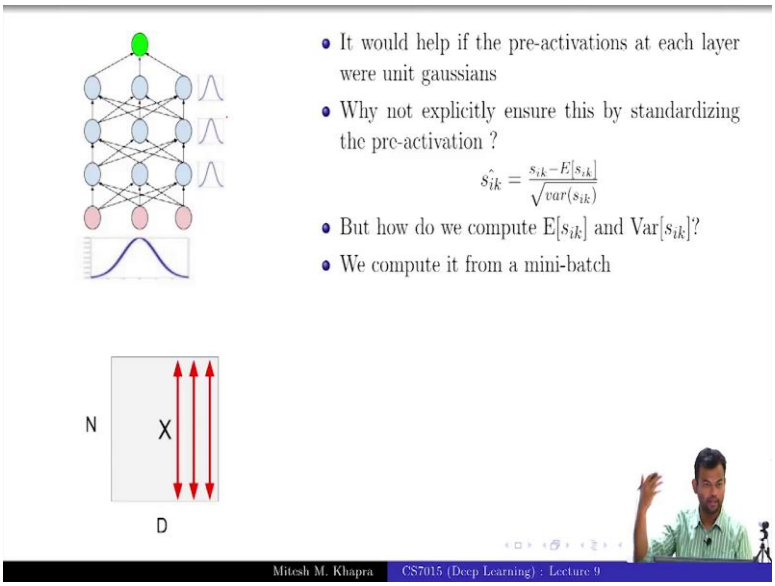


So, to understand the intuition behind this, let us consider a deep neural network ok and let us focus on the last two layers h 4 and h 3; Now, typically will use some mini-batch algorithm for training right, so we will use mini-batch version of gradient descent or mini-batch version of Adam or any of these algorithms, right.

Now, what would happen if there is a constant change in the distribution of h 3 no just think about the question that I am trying to ask you. So, as far as these two layers are concerned h 3 is the input and h 4 is the output it does not matter what has happened so far or in particular does not matter what x was? whether it came from a normal distribution or whatever distribution, right?

At this point my input is h 3 and my output is h 4, now I am training it in mini batches what if across batches my distribution of h 3 looks very different; what would happen? is it a good thing or a bad thing? It is a bad thing, right? So, if you have training data right just think of this as I said just focus on this layer, if you have an input which is not following a fixed distribution is constantly changing during your training then that is always a bad thing right, because you try to adjust to one distribution and now again the distribution is completely changing. So, that always makes our training very very difficult, right? So, if you have a very fluctuant distribution then a training is going to be hard ok, so that is the intuition that I want to build.

(Refer Slide Time: 02:06)



So, now this could actually happen, so it would help if the pre activations at every layer are you need Gaussians because, for the input we made a case that will make the input as unit Gaussian, right?

So, that things are very nice they are all coming all the inputs are coming from the same distribution, but we now realize that at every layer we have an input right it is not that the original input the only input even h 3 is an input even h 4 is an input and so on. So, why not ensure that at every layer your inputs or your h 1, h 2, h 3 also is something, which looks like a Gaussian distribution which comes from a Gaussian distribution. Why not ensure that, that is the basic idea behind batch normalization and how do you do that is the following that you had computed this s i k just as we had done in the derivation earlier right, so s i k is one of these guys.

Now, if you do this what are you actually doing you just normalizing it right you are subtracting the mean and dividing by the variance, so that means you are making it zero mean unit variance and that is the intuition which I was trying to build that why not at every layer have this good distribution which is zero mean unit variance. By even if you are feeding it multiple batches for that batch you will ensure that by this subtraction and division or the normalization process the data will become unit variance and zero mean ok. So, now at every batch the data is coming from the same distribution even if it was

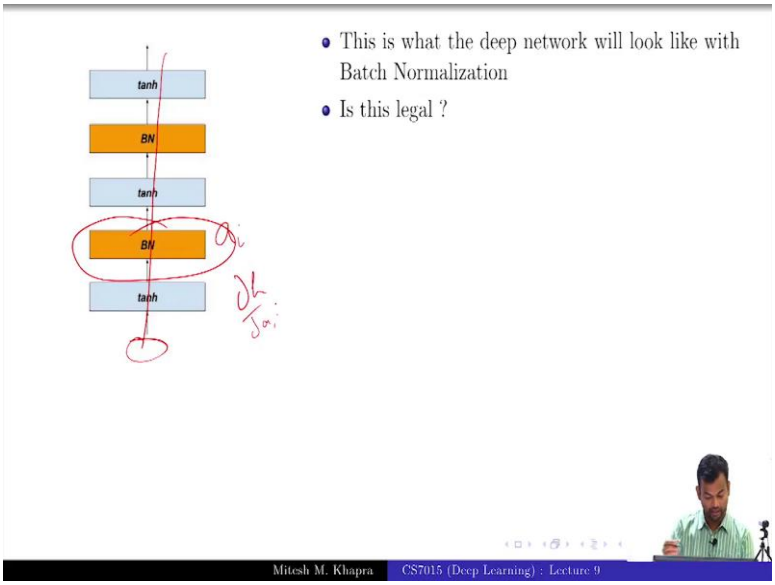
originally from a distant different distribution. But how do we compute this mean and variance?

So, did you understand the question that I am asking I am focusing on this s i k I want to subtract the mean of that s i k, how do I do that? So, the name gives it away batch normalization it cannot be more explicit than that. So, compute the mean for the current batch and the variance for the current batch and normalize your inputs or normalize the S i k according to that you get this. So, now end up with a situation where all your inputs at every layer across different mini-batches seem to come from the same distribution is it fine, the current batch, so you take the average value from the current batch.

So, then it will become zero mean for that batch and unit variance for that batch and this you are ensuring for every batch. So, every independently every batch you are ensuring that it comes from a zero mean unit variance distribution, right? So, overall the effect is that all the batches are coming from the same distribution no. So, at validation time you will compute the mean and variance from your entire data entire training data once after the training is done right.

So, now we will computed from a mini-batch and this is ensure that across mini-batches, now your input always comes from a zero mean unit variance distribution across all the layers.

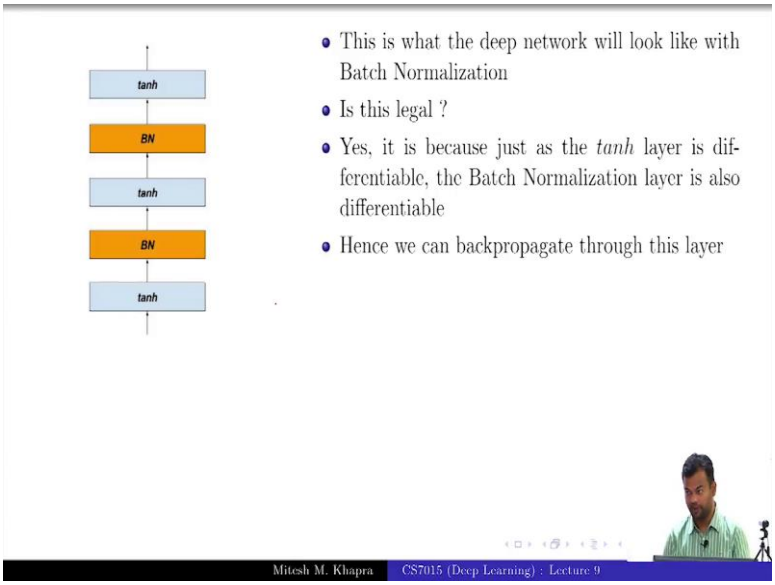
(Refer Slide Time: 05:02)



This is what a deep network will look like with batch normalization right, so what will happen is you passed an input you computed this tan h, then you will have this batch normalization layer watch is what is the operation that the batch normalization is going to do this is the operation that it is going to talk ok. Everyone gets that and now it gives me a unit normalized distribution sorry it gives me a input coming from a zero mean unit variance distribution and then I pass it to the next layer again at a batch normalization layer.

So, after every layer you will actually add a batch normalization here, now my question is, is this legal? What is legal in this course anything that is differentiable, right? So, you have to make sure that if we have added this operation it should be a differentiable operation. So, that you can come so now the gradients have to flow all the way here right, so that means I should be able to compute the gradients with respect to this. So, now this is one of my a i and I should be able to compute dou a i with respect to something or rather the loss dou of the loss function with respect to a i by turns out that the operation, that you have done is actually differentiable.

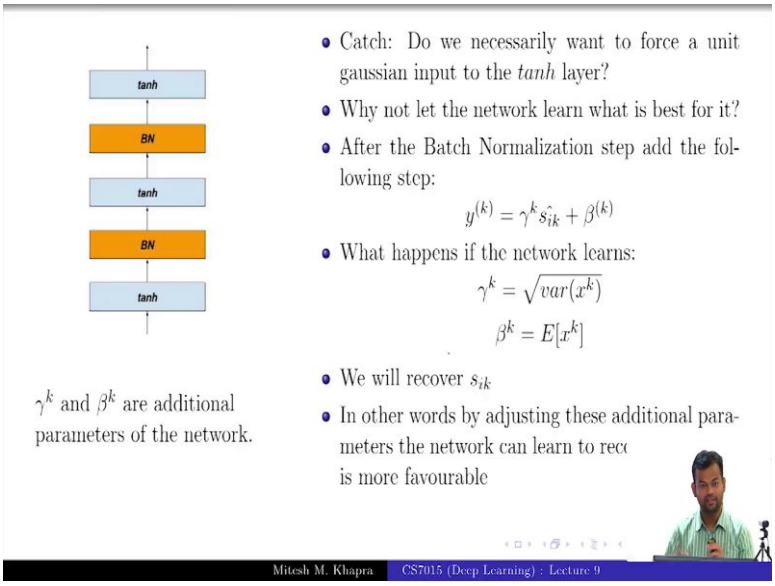
(Refer Slide Time: 06:04)



You can actually work that out and it is not important I am not going to derive it because, it is just yet another derivative that you will take, but it is a you should get the intuition from here right what you are doing is this simple operation and this just looks differentiable, right?

So, the operation that you are doing is differentiable, so that is why you can add these batch normalization layers and you can back propagate through this layer. But now what is the catch here it somehow ties to the question that he was trying to ask, you are actually enforcing that all your are zero mean and unit variance, right? So, this is again some sort of a constraint that you are enforcing right, what if that is not the best situation in which the network can learn; what if to distinguish between some classes it was ok. If the distribution was not same across all the batches they get this, they are enforcing a certain consider they are enforcing a certain condition on all the layers and all of them have to be zero mean and unit variance but that may not always be good.

(Refer Slide Time: 07:02)



So, they do something which is counterproductive, let us see what that is, why not let the network decide what is best for it. So, after the batch normalization layer so this is what a normalized s i k was, after you have done that you compute a y k and this is not the final output this is the output at the k'th layer this is equal to this. Why do they do this and remember that gamma and beta are going to be learnable parameters, what are you doing actually you are again scaling it and shifting it this is the same as adjusting the variance and the mean right.

So, now what happens if the network learns the following you get back the s i k, so you had taken s i k and you had normalized it. But now if you allow these gammas and betas

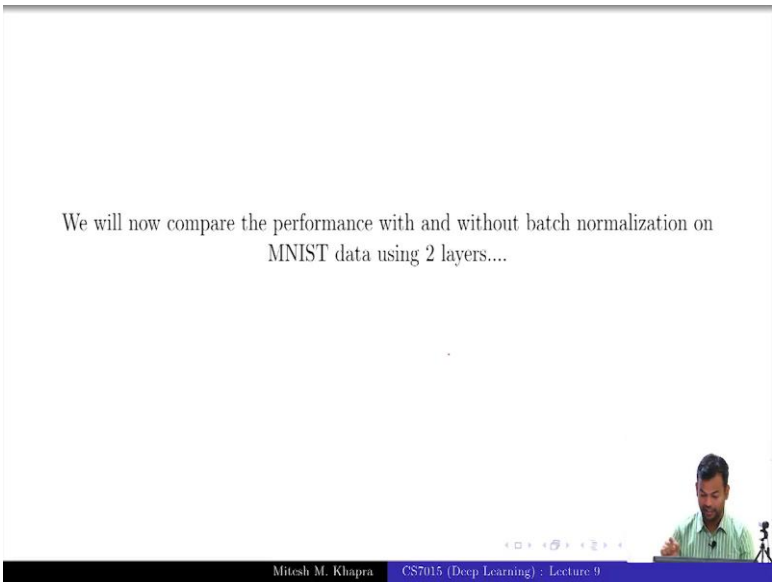
to be there in the network, then the network can decide that maybe at this layer I do not want this normalization I just want to stick to whatever output I was getting.

So, it could learn the gammas and betas in this way and ensure that you get back the unnormalized S, how many of you get this fine lot of you do not seem to get this; but I am pretty sure if you go back and look at it you will get it, right? So, what is happening here is that is why I said it is counterproductive that you first forced it to make at unit mean and 0 variance and now you added no zero mean and unit variance and now you added this operation which is again a scaling and shifting operation. So, remember that when you make the data zero mean and unit variance that is exactly what you do you shift it, so that it become zero mean and you scale it. So, that it becomes unit variance.

So, you are again introducing parameters which again introduce the same flexibility that you could learn gamma and beta in such a way that you could get back the original data which was not normalized ok. So, if the network wants to learn that and if the network fees that is the right thing to do, then it has the flexibility to learn those parameters and you can recover Si.

I think the rationale is that your first making is something which is more standard right and then from there trying to learn it instead of just trying to let it learn in the way. Do you get the difference between the two the first bringing it to all of these things to some standard value, which is between I mean which is the normal distribution and then from there allowing it to learn wherever it has to learn right that is the idea, but it could be the case that the other thing also works here.

(Refer Slide Time: 09:33)



So, now what we will do is we will compare the performance with and without batch normalization on MNIST data using 2 layers.

(Refer Slide Time: 09:40)

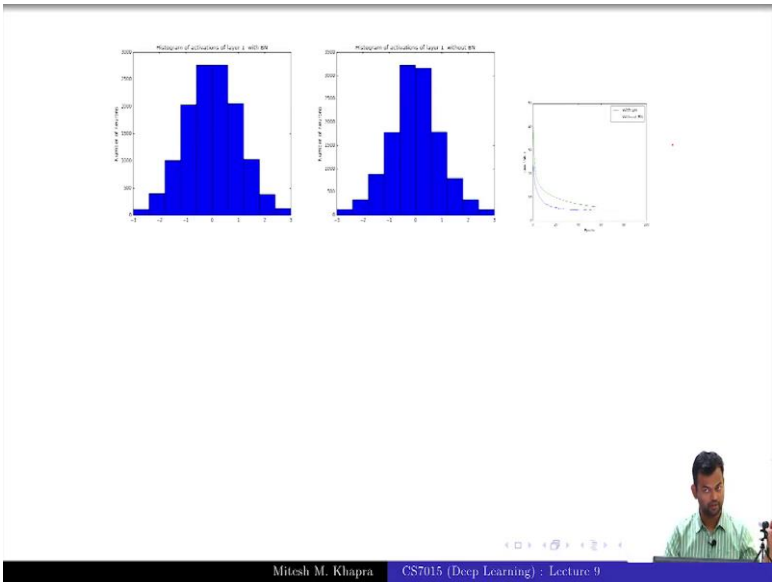


So, here in this figure what I am going to draw is the validation loss, am I no the training loss as I keep increasing the number of epochs and here I am showing you the histogram of the activation functions at layer 1. So, I have trained a deep feed forward neural network and I am showing you what do the activations look like at layer 1 with and without batch normalization. So, remember that we started with this intuition that

without batch normalization there would be this constant fluctuation and the data would seem to come from different distribution at every training instance.

Whereas, with batch normalization you are ensuring at your data comes from zero mean unit variance distribution right and so that is one thing which I want to see another thing I want to see is that how does it affect training, right? So, that is the animation that I am going to show you. So, focus on all these 3 things I do not know how you will do it, but focus on this, focus on this and focus on this with two eyes.

(Refer Slide Time: 10:49)



So, let us see to see what happened right, so this so now look at the focus on the leftmost figure. So, that does not seem to change much with respect to it is mean and variance right, but if you look at the middle figure that is constantly changing it is mean and variance, right? And you see the effect on the training loss that the first one which was with batch normalization, that converges faster as compared to the second one right again an empirical result I am not really proving that this will always happen, this is what empirically we observed.

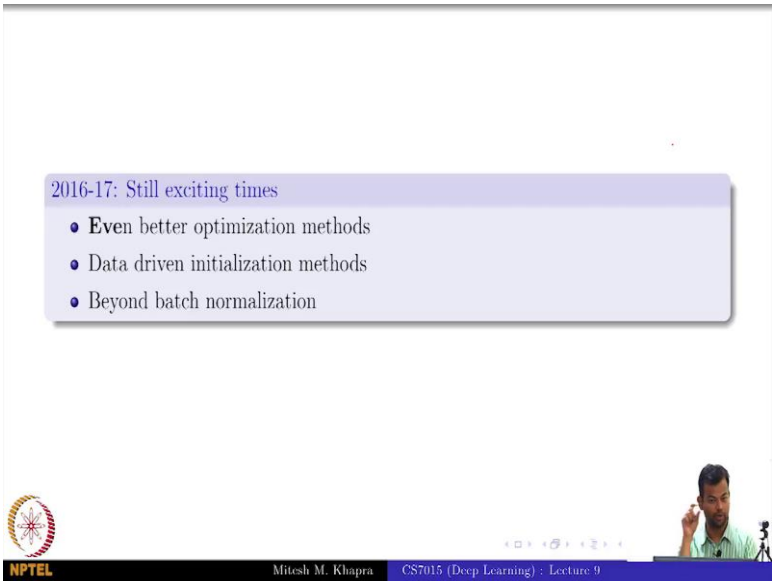
So, this was the story that we covered from 1986 to 2006 where back propagation was already it was already discovered, but was not working well and there was this spark in 2006 that showed that we could do some things to make training really work for deep neural networks, but maybe that something is not sacrosanct. We could try different

things what we tried at that time was unsupervised free training which is almost nonexistent now.

But that lead that led to these thoughts that maybe this is because of optimization, generalization, regularization activation functions and so on, right? So, there was a lot of research in these different areas and that led to a lot of developments which was better optimization algorithms, better regularization, better activation functions, better initializations and batch normalization, right?

So, these a few concepts that you have seen in the past few lectures one being dropout and the other being weight initialization using this Xavier initialization or He initialization and this batch normalization, right? This is something which is all prevalent right, so this is something that you will see in all deep neural networks that get trained definitely in convolutional neural networks and more often than not even in recurrent neural networks. So, these are the two most popular types of neural networks. So, in both of these you will see that these ideas are regularly applied and they always lead to more stable training or better generalization.

(Refer Slide Time: 12:47)



So, now this was all which happened till 2016 or 17 what has happened still since then. So, there is still continuous research in designing better optimization methods, so as I said after Adam there was this Eve which did not become very popular, but there is still

people looking at better optimization methods and there is something which has been developed on Adam and came out in December last year.

Now, people have also started looking at data driven initialization methods right, so instead of having this fixed initialization which is drawn from a unit or just which is drawn from a normal distribution and then just divided by the square root of n. Why not think of data driven initialization methods that, so there are some works on that again not very popular because, most of the shelf things that you will try will not really do any data driven initialization.

But if you really think that you are stuck at some point then you could look at some of these works and see how they try to come up with initializations based on the data that you are dealing with and now after batch normalization there have been some other types of normalizations which have been proposed which seem to work better than batch normalization. But largely the stable configuration which has kind of prevalent is Adam in terms of optimization Xavier or he initialization in terms of initialization ReLU in terms of activation functions. What else is there batch normalization in terms of again regularization plus initialization and dropouts in terms of regularization, right?

So, these are roughly the key terms that you will almost see in all the deeply living deep neural network people that you see right, you will always see when they describe the hyper parameters, they will say that this is how we initialized is this is the drop out that we use this is the batch normalization and the training algorithm more often than not is going to be Adam.

So, they have seen some very crucial elements of training deep neural networks over the past 2 to 3 lectures right and now we will build on these and we will assume that this is what you are going to do. So, now when I talk about neural networks like convolutional neural networks and so on I not go back and tell you use Adam or use batch normalization or assume that you already know these things and you will try to train your networks using these tricks that we have your learned.

The last couple of lectures have been about tips and tricks for deep neural networks and from here on in the next lecture will move on to what word2Vec, because that is what you need for your assignment. So, in the next lecture we will do a word representations,

so that is essentially seeing an application of feed forward neural networks and from there on we will move on to convolutional neural network.